

이 편집기의 AI 조수는 어떤 식으로 작성하는가

채팅창에 넣은 한 마디가 실제 게임 데이터가 되기까지,
무엇이 어떤 순서로 일어나는지를 소스 코드 근거로 쉬운 말로 풀어 씁니다.
함수 이름은 원문 그대로 두었으니 코드를 찾아볼 때 그대로 쓰시면 됩니다.

대상 rpg-zzu 편집기 (OPRN Studio) · AI 조수 전체 계통

기준 소스 HEAD `c56628804` — 조사 중 `72f1f179` 로 이동했으나 `git diff -- src` 결과 차이 없음

조사 규모 출처 원장 64항목(파일 묶음·검증 기록 포함) · 편집기 AI 패널 다수 · 병렬 조사원 13명 · 반론 6라운드

실측 검증 도구 레지스트리 직접 import(exit 0) · 압축 함수 실행 · 편집기 실제 화면 31장 캡처

한계 실제 모델 호출 없음 — 화면 속 대화·계획·변경은 실제 렌더러에 표시한 로컬 예시 데이터

작성 2026년 09월 09일

이 문서를 읽는 법

이 편집기의 AI 조수는 겉보기에 "대화하면 맵이 바뀌는" 기능이지만, 내부는 여러 겹의 확인 절차로 짜여 있습니다. 그 절차를 모르면 "왜 AI가 안 물어보고 바꿨지?", "적용됐다는데 왜 저장이 안 됐지?" 같은 일이 이해되지 않습니다. 이 문서는 그 절차를 순서대로 보여 줍니다.

목차

- 1 한 장으로 보는 전체 흐름 — 여덟 단계
- 2 AI를 부르는 길은 하나가 아니다 (3층 구조)
- 3 1단계 · 사람이 한 줄을 넣는다
- 4 2단계 · 무슨 뜻인지 먼저 분류한다
- 5 3단계 · 큰 일이면 계획을 세운다
- 6 4단계 · 지금 상황을 설명서로 만들어 붙인다
- 7 5단계 · 여기가 실제 "작성" — 생각하고 도구 쓰기의 반복
- 8 AI가 쓸 수 있는 도구 205개
- 9 6단계 · 다른 심판이 검사한다 (사람이 아니다)
- 10 "승인"이라는 말의 세 가지 뜻 — 가장 오해가 큰 부분
- 11 7단계 · 실제 게임에 반영한다
- 12 8단계 · 저장됐는지 따로 증명한다
- 13 분량 제한 — 네 겹의 서로 다른 예산
- 14 AI는 무엇을 기억하는가
- 15 설정한 값이 그대로 가지 않는 구간
- 16 실제 화면 모음
- 17 부록 — 함수 이름 색인 · 문서와 코드의 불일치 · 확정하지 못한 것 · 조사 방법

먼저 알아 둘 세 가지

1. 일반 채팅의 쓰기 작업은 사본에서 시작합니다. `cloneDetachedDraft` 로 사본을 만들고 검수 후 반영합니다. 별도 생성 화면의 경로는 2절에서 구분합니다.
2. 일반 채팅에는 "적용할까요?" 버튼이 없습니다. 그 자리를 별도 검수 호출의 검사가 대신합니다. 사람이 누르는 버튼은 일부 화면에만 남아 있습니다.
3. "적용됨"은 "저장됨"이 아닙니다. 서버에 실제로 남았는지는 `proveAppliedRevision` 이 다시 읽어와 확인해야 확정됩니다.

표기 약속

본문은 쉬운 말로 쓰고, `함수 이름` 과 파일 위치는 원문 그대로 둡니다. `파일.ts:123` 은 그 파일 123번째 줄을 뜻합니다. 수치에는 출처를 붙였습니다 — `실측` 은 코드를 직접 돌려 얻은 값, `추정` 은 저자가 분류한 값, `계산` 은 실측값으로 계산한 값입니다.

1. 한 장으로 보는 전체 흐름

다음 여덟 단계는 일반 채팅에서 실제 작성을 요청했을 때의 기본 흐름입니다. 질문·계획 모드에서는 일부 단계만 거칩니다. 다음 그림이 이 문서 전체의 지도입니다.

① 사람이 입력한다

채팅창에 한 줄
`send / sendText`

② 무슨 뜻인지 먼저 분류

질문? 계획? 작업?
`createLlmIntentDeclarer`

③ 큰 일이면 계획을 세운다

층·항목으로 쪼갬다
`runOrchestratorPlanner`

④ 설명서를 만들어 붙인다

지금 프로젝트 상황
`buildSystemPrompt`

⑤ 실제 "작성" — 생각하고 도구를 쓰는 반복

`executeTurnLoop` — `assistantSession.ts:4230`

모델에게 묻는다 → 도구를 실행한다 → 사본에만 쓴다

`chatCompletion` → `runTool` → `cloneDetachedDraft`

결과를 되돌려준다 → 부족하면 다시 한 바퀴

`toolResultForModel` → `roundCap`

⑥ 다른 심판이 검사한다 (사람 아님)

별도 모델이 결과만 보고 판정

`reviewCurrentDraft` → `buildIndependentReviewRequest`

⑦ 통과하면 실제 게임에 반영

사람 승인 카드 없이 바로 적용

`applyProposedProject` → `store.replace`

⑧ 저장했는지 따로 증명한다

적용 성공 ≠ 저장 성공 · 다시 읽어와 내용이 같은지 확인

`proveAppliedRevision` → `verified=true`

승인이 안 되면 고칠 점을 알려 주고 ⑤로 되돌린다

그림 1. 한 마디가 게임 데이터가 되기까지의 여덟 단계. ⑤번 점선 상자가 실제 작성이 일어나는 반복 구간이며, 주황 화살표는 다시 시키는 경로입니다. 파란 등쪽 글씨는 소스 코드의 실제 함수 이름입니다.

한 문장 요약

일반 채팅의 쓰기 작업은 **사본에 도구로** 변경을 만들고, **별도 검수 호출을 통과한 변경만** 실제 프로젝트에 반영합니다. 서버에 저장됐는지는 별도로 확인합니다. 사람에게 물어보는 대신 기계가 검사한다 — 이것이 이 설계의 핵심 선택입니다.

2. AI를 부르는 길은 하나가 아니다

"편집기의 모든 AI 요청은 조수 세션을 지나간다"고 생각하기 쉽지만 **사실이 아닙니다**. 모델을 직접 부르는 길도 있습니다. 그 길은 일반 채팅의 작업 계획·독립 검수를 거치지 않으며, 화면별 검증·사용자 승인 방식이 따로 있습니다. 세 층으로 나눠 봐야 합니다.

1층 · 일반 채팅 (주 경로)

일반 채팅의 계획·검수·적용·저장증명 경로

`renderAiChatPanel(:290) → send(:1677) → sendText(:1527) → executeTurn(:1649) → AssistantSession.sendMessage(:1686)`

맵·이벤트·데이터베이스·이미지·문서 — 무엇이든 저작 가능. 데이터베이스 AI 바도 브리지로 이 길에 합류합니다

`sendAiAssistantMessage` — `aiAssistantBridge.ts:106`

2층 · 범위 한정 세션 어댑터

자기 세션을 따로 띄우지만 결국 같은 `sendMessage`로 들어옴

`runRegionTask(:529 → :728) · clusterAiModal(:187 → :296)`

선택 사각형 작업, 클러스터 편집. 여기에는 적용/폐기 또는 수락/거부 버튼이 남아 있습니다

주의: 선택 경로는 세션 진입 전에 의도 분류용 모델 호출을 먼저 합니다 (`aiRegionTaskRunner.ts:67`)

3층 · 모델 직접 호출 유틸

`chatCompletion` 직접 호출 — 일반 채팅의 계획·독립 검수는 거치지 않음

`generateDatabaseRecordWithAi(aiDatabaseGeneration.ts:229,:243) ·`

`runEventCommandAssist(eventCommandAssist.ts:620)`

`requestCpenTilesetMapping(tilesetAiCpenClient.ts:35) ·`

`structureKitEditorDialog.ts:2330 · createLlmIntentDeclarer`

데이터베이스 생성 대화상자는 만든 레코드를 곧바로 목록에 넣습니다(검수 단계 없음). 이벤트 보조는 사람이 골라야 반영됩니다

그림 2. AI 진입 경로의 3층 구조. 1층은 일반 채팅의 흐름이고, 2층은 같은 세션 엔진을 범위 작업에 맞게 사용합니다. 3층은 조수 세션을 건너뛰고 `chatCompletion` 을 바로 부릅니다.

왜 이 구분이 중요한가

예를 들어 데이터베이스의 "AI로 몬스터 만들기" 대화상자는 3층입니다. `generateDatabaseRecordWithAi` `aiDatabaseGeneration.ts:229, :243` 가 모델을 직접 부르고, 돌아온 레코드를 곧바로 목록에 등록합니다. 독립 검수 단계가 없습니다. 반면 이벤트 편집기의 AI 보조는 같은 3층이지만 사람이 체크박스 고른 뒤 버튼을 눌러야 반영됩니다 (`runEventCommandAssist` `eventCommandAssist.ts:620`). 즉 "AI 기능"이라는 한 단어로 묶어 이해하면 어느 쪽인지 알 수 없습니다.

층	해당 경로	특징
1층 일반 채팅	<code>renderAiChatPanel</code> → <code>send</code> → <code>sendText</code> → <code>AssistantSession.sendMessage</code>	계획·독립 검수·적용·저장 증명을 모두 갖춘 완전한 길. 데이터베이스 AI 바도 <code>sendAiAssistantMessage</code> 브리지로 여기 합류
2층 범위 한정	<code>runRegionTask</code> , <code>clusterAiModal</code>	자기 세션을 띄우지만 결국 같은 <code>sendMessage</code> 로 들어옴. 적용/폐기 또는 수락/거부 버튼이 남아 있음
3층 직접 호출	<code>generateDatabaseRecordWithAi</code> , <code>runEventCommandAssist</code> , <code>requestOpenTilesetMapping</code> , <code>createLlmIntentDeclarer</code>	<code>chatCompletion</code> 을 바로 부름. 작업 계획도 독립 검수도 없음

주의 — 두 가지 예외

재시도 버튼은 `retryLastTurn` `assistantSession.ts:3375` → `:3390` 으로 `sendMessage` 를 건너뛰고 `runTurnLoop` 에 바로 다시 들어갑니다.
영역 선택 후 보내기는 조수 세션에 들어가기 전에 의도 분류용 모델 호출을 한 번 먼저 합니다 `aiRegionTaskRunner.ts:67` → `intentDeclarationClient.ts:134`.

3. 1단계 · 사람이 한 줄을 넣는다

채팅창에서 Enter를 누르거나 보내기 버튼을 누르면 같은 함수 하나로 모입니다. `send` `aiChatPanel.ts:1677` 입니다. 이 함수는 파일 외부로 내보내지 않는 내부 함수이므로 다른 파일에서 직접 부를 수 없습니다. 패널 전체를 그리는 공개 함수는 `renderAiChatPanel` `aiChatPanel.ts:290`입니다. 이 파일에는 다른 공개 함수도 있습니다.

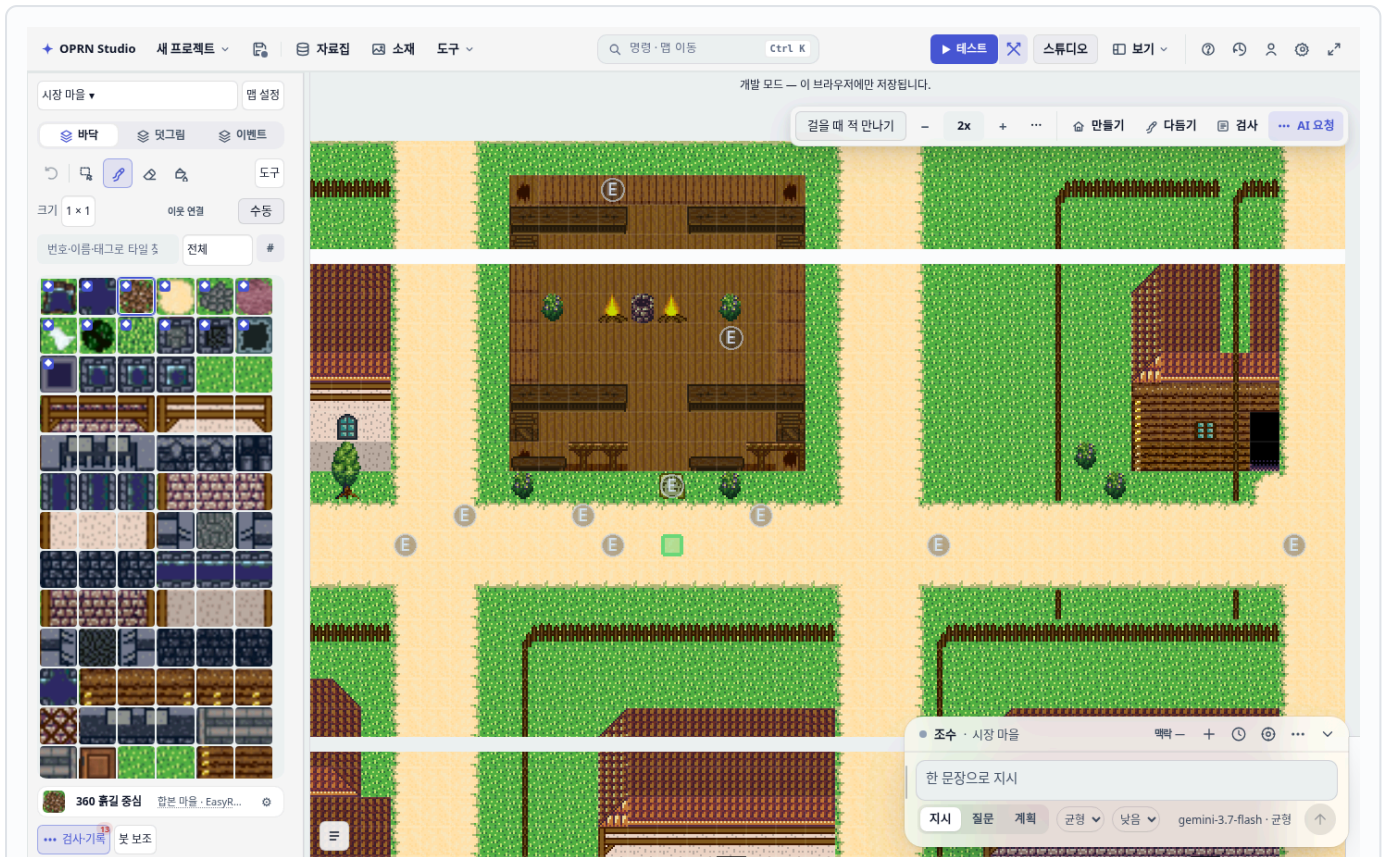


그림 3. 편집기에서 AI 조수를 연 실제 화면. 오른쪽 아래가 조수 독이고 '한 문장으로 지시' 입력칸 아래에 지시·질문·계획 세 모드, 자율 수준 (균형), 추론 강도(낮음), 사용 모델(gemini-3.7-flash)이 함께 표시됩니다.

보내기 전에 하는 준비

- 선택 영역이 있는지 확인 — 맵에서 사각형을 선택해 둔 상태면 `sendSelectionRegionTask` :1665 로 갈라져 영역 전용 경로를 탐니다.
- 범위 판정 — `resolveTurnScope` :1502 가 그 선택이 이번 작업 범위인지 결정합니다.
- 맥락 꼬리표 붙이기 — `contextFooter` :1190 가 현재 맵 이름과 재료 힌트를 원문 뒤에 붙입니다. 모델에게는 이 꼬리표가 함께 가지만, 의도를 분석할 때는 `stripContextFooter` `contextFooter.ts:33` 로 떼어냅니다.
- 이미 작업 중이면 줄 세우기 — `pendingSends` :1214 에 넣고 `drainPendingSends` :1222 가 먼저 온 것부터 처리합니다.

준비가 끝나면 `sendText` :1527 → `executeTurn` :1649 를 거쳐 `AssistantSession.sendMessage` `assistantSession.ts:1686` 로 넘어갑니다. 이 `sendMessage` 가 조수 세션의 유일한 사용자 메시지 입구입니다.

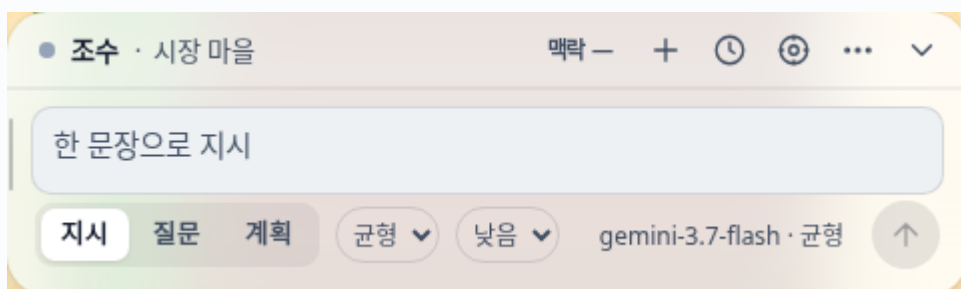


그림 4. 입력 영역 확대. 왼쪽부터 모드(지시/질문/계획), 자율 수준, 추론 강도, 작업 대상 맵. 이 값들이 `sendMessage` 의 옵션으로 함께 넘어갑니다.

4. 2단계 · 무슨 뜻인지 먼저 분류한다

본격적으로 일을 시키기 전에 **실행 모델(liteModel)**이 먼저 판단합니다. "이건 질문인가, 계획만 세우라는 건가, 실제로 만들라는 건가?"

담당은 `createLlmIntentDeclarer` `intentDeclarationClient.ts:89` 이고 결과를 해석하는 것은 `parseIntentDeclaration` `intentDeclaration.ts:267` 입니다. 결과는 자유 문장이 아니라 **정해진 항목의 JSON**입니다 — 모드, 작업 공간, 대상 맵, 선택 영역 사용 여부, 계획 필요 여부, 그리고 "쓰기 전에 무엇을 읽어야 하는가" 같은 조건들입니다.

여기서 아껴 쓰는 것

빈 요청이거나 "계속" 같은 이어가기 요청이면 이 모델 호출을 **건너뛵니다** `intentDeclarationClient.ts:98-100`. 분류에 넘기는 정보도 실제 맵·선택·도구·위키·시작 위치뿐입니다(`buildIntentFacts` :56).

작업 요청으로 판정되면 별도로 **요청문에서 지켜야 할 약속을 뽑아내는** 검사가 한 번 더 실행됩니다

(`REQUEST_COVERAGE_AUDIT` `requestCoverage.ts:12`, `parseRequestCoverageResult` :33). 이때 원문에서 인용되지 않은 구간은 "아직 해결되지 않은 요구"로 남겨 둡니다 :69-74 — 사용자가 시킨 것을 조용히 빠뜨리는 일을 막는 장치입니다.

모드에 따라 달라지는 것

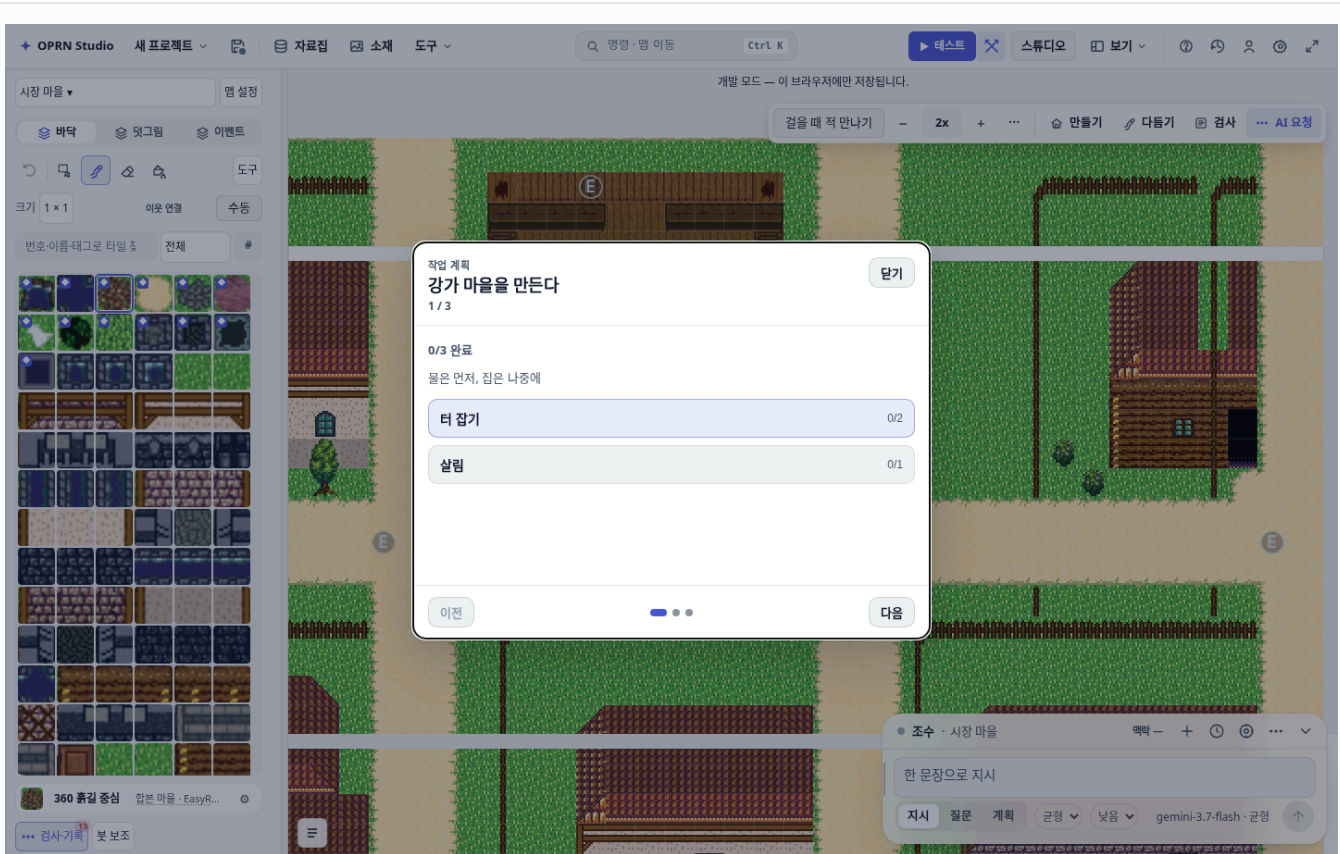
모드	동작
질문	쓰기 도구를 아예 목록에서 빼고(<code>isWriteToolName</code> <code>assistantSession.ts:671</code>) 실행 단계에서도 다시 거부합니다 :4574-4577. 답의 점진적 표시를 요청하지만 실제 표시 방식은 전송 경로에 달려 있습니다(15절 참조).
계획	계획만 작성하고 <code>finishPlanOnlyTurn</code> :2219 으로 멈춰 "계속"을 기다립니다.
지시	계획 → 도구 실행 → 검수 → 적용의 전체 과정을 진행합니다.

알아 두면 좋은 차이 — 글자가 흐르는 때와 안 흐르는 때

질문 모드에는 글자를 점진적으로 표시하는 코드가 있지만, 조사한 **companion** 경로는 비스트리밍 응답을 반환합니다. 따라서 이 경로에서 글자가 한 자씩 나온다고 보장할 수 없습니다. 지시 모드에서는 작업자의 중간 설명을 내보내지 않습니다. `executeTurnLoop` 이 실시간 전송 여부를 질문 모드일 때만 켜기 때문입니다 `assistantSession.ts:4303-4304`. 대신 도구 실행 상황이 실시간으로 보이고, 최종 설명은 독립 검수가 끝난 뒤 한 번에 나타납니다.

5. 3단계 · 큰 일이면 계획을 세운다

"마을을 만들어줘" 같은 큰 요청은 한 번에 처리하지 않습니다. 주 모델이 **도구 없이** 한 번 호출되어 층(layer)과 항목(item)으로 된 작업 계획을 만듭니다. 지침은 `ORCHESTRATOR_SYSTEM_PROMPT` `workPlan.ts:133-155` 에 있고, 결정은 직접 실행 / 이어가기 / 새 계획 / 계획 수정 넷 중 하나입니다.



실제 UI · 로컬 예시 데이터 그림 5. 작업 계획 modal의 실제 화면 (계획 내용은 저장소 테스트 예시) (`openWorkPlanBook`

`aiWorkPlanModal.ts:57`). '강가 마을을 만든다'는 목표 아래 '터 잡기 0/2', '살림 0/1' 같은 총과 항목이 보이고, '물은 먼저, 집은 나중에' 같은 순서 메모가 함께 표시됩니다.

계획 항목은 함부로 완료되지 않는다

각 항목에는 `instruction` (무엇을 할지), `doneWhen` (무엇이 되면 끝인지), `successTools` (반드시 성공해야 하는 도구)가 붙습니다. `advanceWorkPlanFromTools` `workPlan.ts:801` 는 필수 도구가 전부 성공했는지와 산출물이 실제로 생겼는지를 함께 봅니다. `canCompleteWorkItem` :833 과 `completeWorkItemById` :873 는 다른 항목의 성공 기록을 가져다 쓰는 것을 막습니다.

한계 — `doneWhen` 은 기계가 판정하는 식이 아니다

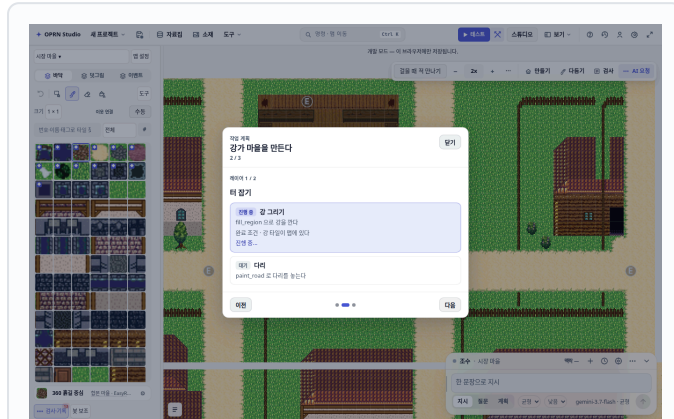
`doneWhen` 에 적힌 문장은 사람이 읽는 조건 설명이며, 그 문장을 자동으로 검증하는 장치는 없습니다. 실제 판정은 필수 도구 성공 여부와 뒤에 나오는 인수검사가 담당합니다.

중간에 멈추지 않게 하는 장치

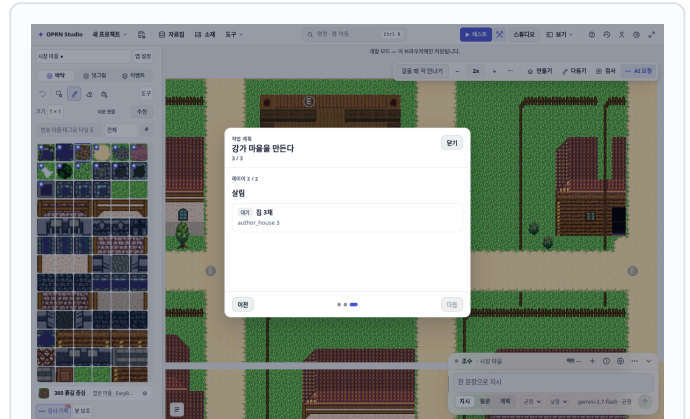
모델이 일을 덜 하고 멈추려 할 때 다시 밀어 주는 구조가 있습니다. `ralphContinuationDecision` `workPlan.ts:763` 와 `shouldRalphContinue` :788 가 판단하고 `formatRalphContinueMessage` :716 가 이어갈 말을 만듭니다. 다만 무한히 밀지는 않습니다 — 한 턴에 최대 256회 :193, 같은 항목에서 진전이 없으면 3회 :204, 같은 대상에서 같은 문제로 4회 실패하면 막힌 것으로 처리합니다 `assistantSession.ts:226`.

계획과 다른 것 — 공간 밑그림(BuildSpec)

계획이 "무엇을 언제"라면 `BuildSpec` `buildSpec.ts` 은 "어디에 얼마나"입니다. `validateBuildSpec` :158 이 맵 경계·크기·겹침·기존 내용 보호를 검사하고, 지우려는 자리에 기존 구조물이 6칸 이상이면 `confirmDestroy` 를 요구하고 :235-246, 배치 주변에 기존 내용이 4칸 이상이면 지울지 유지할지 명시하게 합니다 :250-257. 사용자가 영역을 선택했다면 그것이 곧 암묵적 밑그림이 되며 (`implicitSpecFromScope` :551), 그 안을 덮어쓸 권한이지 맵 전체 권한은 아닙니다.



실제 UI · 로컬 예시 데이터 그림 6. 작업 계획 1층 상세 — 항목별 진행 상태.



실제 UI · 로컬 예시 데이터 그림 7. 작업 계획 2층 — 층별로 페이지가 넘어갑니다.

6. 4단계 · 지금 상황을 설명서로 만들어 붙인다

모델은 프로젝트 파일을 마음대로 열어 볼 수 없습니다. 그래서 매번 지금 상황 설명서를 만들어 앞에 붙입니다. 담당은 `buildSystemPrompt` `contextBuilder.ts:693` 입니다.

설명서에 들어가는 것 (순서대로)

1. 정체성과 작업 수칙 — 편집기 조수라는 설명, `AGENT_UX_POLICY_LINES` `promptPolicies.ts:5`, 도구 라우팅 안내, 24개 작업 수칙 `contextBuilder.ts:133-213`
2. 도구 색인과 작업 레시피 — `buildToolCapabilityIndex()`, `buildTaskRecipes()` `toolCapabilityIndex.ts:59,:92`, 이벤트 페이지 문법, 사람 성향 기억. 이 묶음은 예산 밖에서 보호됩니다(`withFixedBlocks` `contextBuilder.ts:771`)
3. 프로젝트 요약 — 제목, 시작 맵, 모든 맵의 크기와 이벤트 수, 데이터베이스 항목 개수, 이름 붙은 스위치·변수 `queryTools.ts:62-95`. 데이터베이스 레코드 전문이나 모든 이벤트 페이지는 여기 없습니다
4. 타일 지식과 어휘 — 사람이 가르친 타일 설명(타일셋당 20개, 설명 80자), 승인된 그룹의 역할·배치 규칙 :269,:357
5. 현재 보고 있는 맵 — 화면에 보이는 영역을 문자 격자로. `#` 은 막힘, `.` 은 통행 가능, `~` 은 물, `T` 는 나무, `E` 는 이벤트 `queryTools.ts:140-217`
6. 보호 블록 — 뷰포트 정보, 마을 설계 계약, 관련 위키 문서, 세계관, 프로젝트 감독 지침 `contextBuilder.ts:733-750`

중요 — 넘치면 우선순위로 버리는 게 아니라 그냥 자릅니다

현재 구현은 조립한 문자열을 `assembled.slice(0, budget)` 으로 단순 절단합니다 `contextBuilder.ts:736`. 문장이나 JSON 경계를 맞춰 자르지 않으므로 JSON 중간이 끊길 수 있습니다. 또 뒤에 붙는 보호 블록 때문에 최종 길이가 예산을 넘을 수도 있습니다. 코드 주석에 남아 있는 "색선별 우선순위 폐기" 설명은 현재 동작과 다릅니다.

요약만 보는 게 아니다 — 원본도 따로 실린다

많은 사람이 "시"는 요약만 본다"고 생각하지만 사실이 아닙니다. `extractOriginalContext` `originalContext.ts:117` 가 편집 전 사본에서 대상 맵 전체 영역, 각 이벤트의 완전한 페이지 트리, 타일 배열, 타일셋 메타를 담은 원본 묶음을 만듭니다.

`OriginalContextStore.message` :347 는 남은 자리에 완전한 항목 단위 JSON만 넣고, 자리가 부족한 항목은 넣지 않는 대신 "필요하면 `get_original_context` 로 더 읽으라"는 목록을 남깁니다. 잘린 JSON을 넣는 일은 없습니다.

그리고 실제로 전달된 것만 "읽었다"고 인정합니다 — `observeDelivered` :297. 도구를 불러 놓기만 하고 결과가 모델에 안 갔다면 읽은 것으로 치지 않습니다.

7. 5단계 · 여기가 실제 "작성"이다

지금까지가 준비였고, 여기서부터가 진짜 작성입니다. 모델에게 묻고 → 도구를 실행하고 → 결과를 되돌려주고 → 부족하면 다시 묻는 반복입니다. 담당은 `executeTurnLoop` `assistantSession.ts:4230` 이고 반복문은 :4256 의 `for` (`let round=0; round<roundCap; round+=1`) 입니다.

가장 흔한 오해 — 한 바퀴는 도구 하나가 아닙니다

한 바퀴 = 모델 응답 한 번입니다. 모델이 한 번에 도구 다섯 개를 부르면 그 다섯 개가 한 바퀴 안에서 모두 실행됩니다. 그래서 "16바퀴"가 "도구 16개"를 뜻하지 않습니다. 독립 검수도 한 바퀴를 소비합니다 :4476.

한 바퀴 안에서 벌어지는 일

1 도구 목록을 만든다

활성 도구 205개 전부 + 원본 조회 + 밑그림 + (조건부) 계획 4종 + 인수검사 2종 :4270-4278. 질문 모드면 쓰기 도구를 뺍니다. 옛 문서가 말하는 "40개만 보여준다"는 제한은 현재 코드에 없습니다.

2 필요하면 대화를 압축한다

`maybeCompactConversation` :3533, 실행 :4286. 사용자 한 턴에 최대 1회.

3 요청을 조립하고 한계를 검사한다

`buildGroundedRequest` `contextBuilder.ts:663`. 도구 스키마 분량과 예약분까지 세어 넘치면 도구를 잘라 억지로 통과시키는 대신 `original-context-window-exceeded` 오류를 냅니다 :685-687.

4 모델에게 묻는다

`chatWithTransientRetry` `assistantSession.ts:4149` → `chatCompletion` `llmClient.ts:791`. 일시적 오류면 세션이 3회까지 다시 시도합니다.

5 도구 호출을 해석한다

`parseToolCall` :5250. JSON이 깨졌으면 실행하지 않고 "인수 JSON이 잘못됐다"고 되돌려 줍니다 :5269.

6 도구를 실행한다 — 사본에

`runTool` `toolRunner.ts:70` → `runToolDefinition` :84 → 각 도구의 `run`. 쓰기 도구는 매번 `createDraft` 로 임시 사본을 또 만들고, 집 보호 검사와 `commitChangeset` 을 통과한 뒤에만 사본에 반영합니다 :126-162.

7 결과를 모델에게 되돌려준다

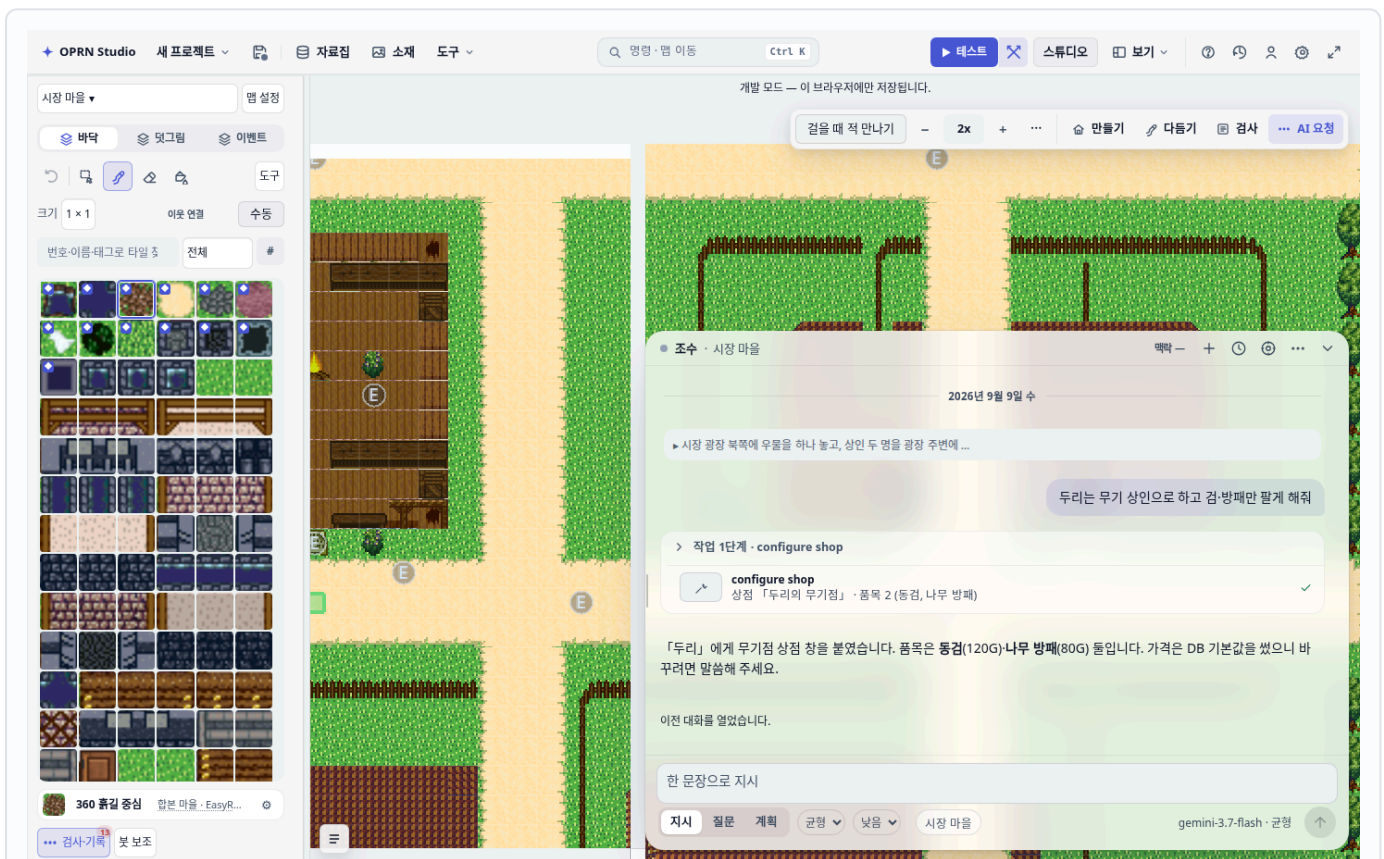
`toolResultForModel` `assistantSession.ts:5188` 이 성공 여부·요약·변경 내역·문제·경고·데이터를 넘기고, `compactToolDataForModel` :5202 가 큰 타일 배열을 줄입니다.

8 지도를 봐야 하면 그림을 첨부한다

시각 도구 결과는 캔버스로 그려(`renderToolImages` `toolImageRenderer.ts:54`) 별도 메시지로 붙입니다 :4802-4813, :4831-4843.

9 아직 할 일이 남았으면 다시 한 바퀴

계획 미완료·인수검사 미충족·분량 미달이면 이어갈 말을 넣고 반복합니다.



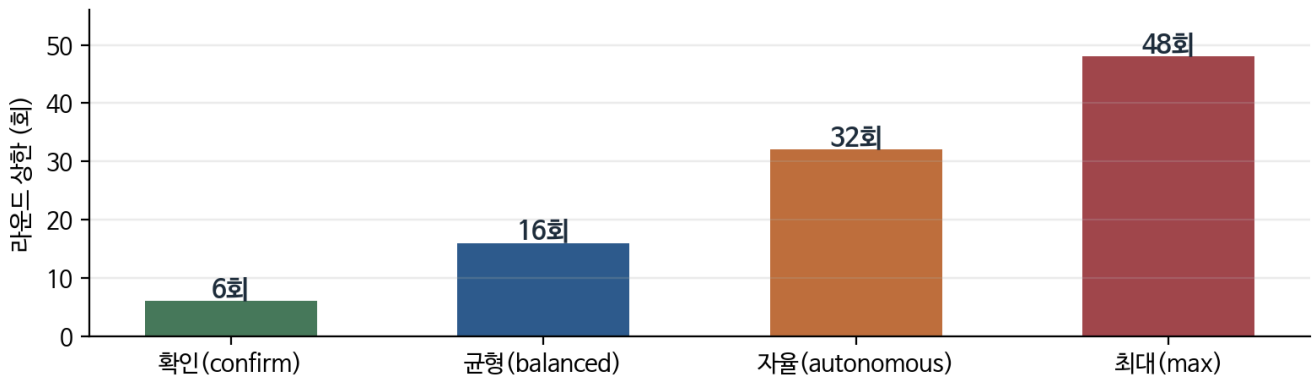
실제 UI · 로컬 예시 데이터 그림 8. 도구 실행이 보이는 실제 화면 (대화 내용은 로컬 예시 데이터). '작업 1단계 · configure shop' 아래에 실행된 도구와 요약('상점 「두리의 무기점」 · 품목 2')이 체크 표시와 함께 나타나고, 그 아래 최종 설명이 붙습니다.

몇 바퀴까지 도는가

실제 상한은 `min(config.maxToolCalls, resolveAutonomy(level).budgetCap)` `assistantSession.ts:4251-4252` 입니다.

기본값은 `maxToolCalls` 2000 `llmClient.ts:91` 과 균형 수준의 16 `autonomyLevels.ts:34` 중 작은 값이라 실질 16바퀴입니다.

자율 수준별 한 턴 안에서 모델이 생각할 수 있는 횟수 (단위: 회)



근거: autonomyLevels.ts:16 AUTONOMY_LEVELS budgetCap / 실제 상한 = min(maxToolCalls, budgetCap) (assistantSession.ts:4252)

그림 9. 자율 수준별 한 턴 안 라운드 상한. 실측 AUTONOMY_LEVELS autonomyLevels.ts:16

그 바깥에 별도의 자동 계속 예산이 또 있습니다. `runAutonomousDriver` :1785 가 첫 턴이 끝난 뒤 "계속"을 스스로 보내며, 최대 48턴을 추가로 실행합니다(`AGENT_RUN_MAX_TOTAL_STEPS` :660, 사용 :1826).

화면에 보이는 숫자와 실제 상한은 다릅니다

컴포저에 표시되는 예산 분모(`runBudgetTotal` aiChatPanel.ts:1261)는 자율 수준의 16/32/48을 그대로 보여 줍니다. 그러나 바깥 자동 계속 드라이버의 상한은 언제나 48입니다. 진행 표시만 보고 실행 한계를 설명하면 틀립니다.

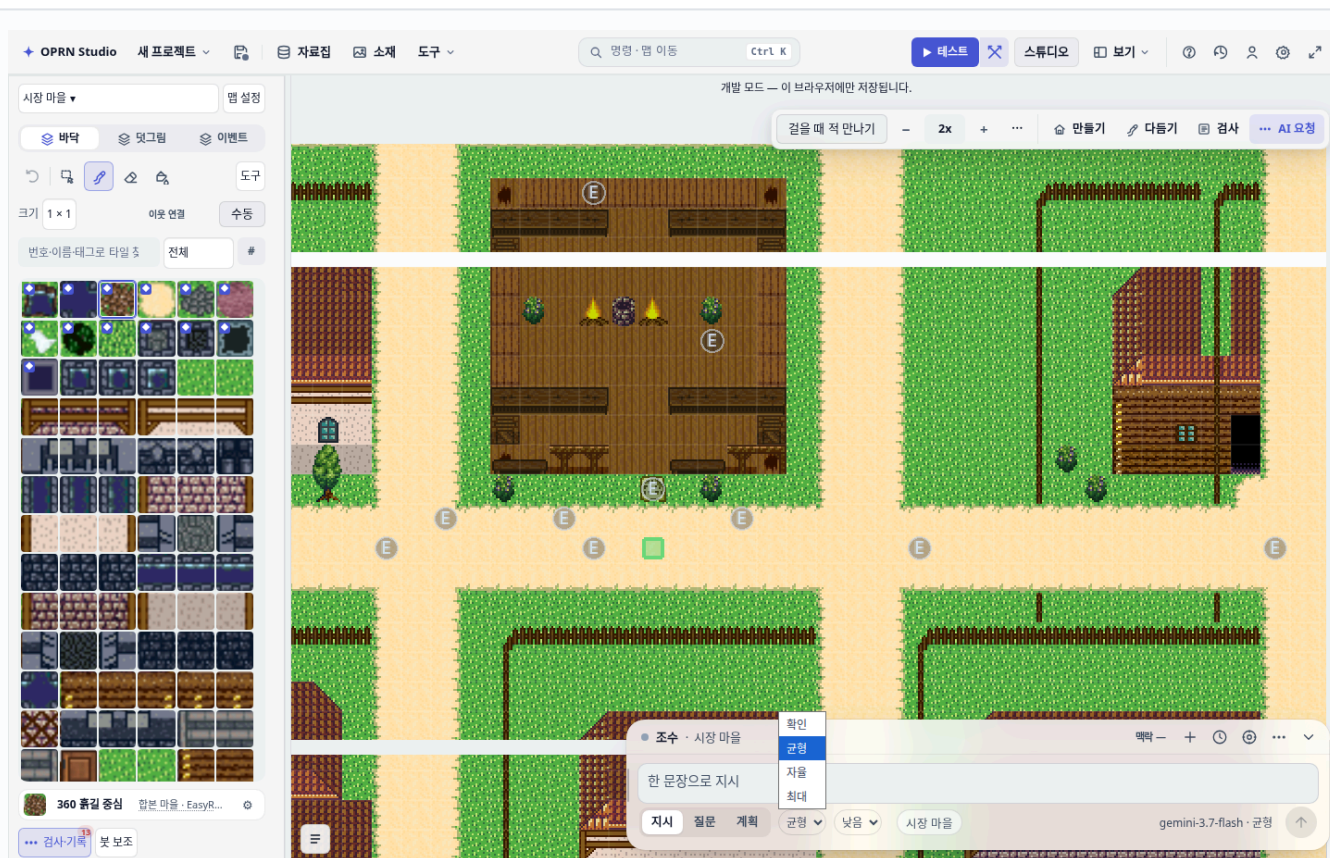
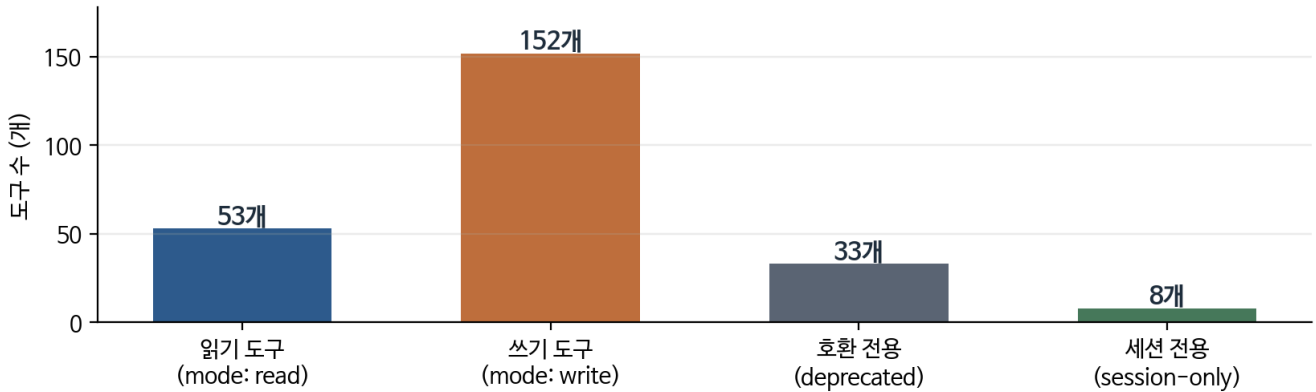


그림 10. 자율 수준 선택 실제 화면. '확인'을 고르면 계획만 세우고 멈춥니다.

8. AI가 쓸 수 있는 도구 205개

AI는 문장으로 맵을 바꾸지 않고 정해진 함수를 호출해서 바꿉니다. 그 함수 목록이 도구 레지스트리입니다. 정보는 `TOOL_REGISTRY` `toolRegistry.ts:156`이며, 모델에게 보여줄 형태로 바꾸는 것은 `toOpenAiTools` :266 입니다.

AI 조수가 쓸 수 있는 도구 구성 (단위: 개)



실측(registry 직접 import): 238개 등록 = 활성 205(읽기 53 / 쓰기 152) + 호환 전용 33. 세션 전용 8개는 registry 밖. 완전히 활성화된 쓰기 툴에서 모델에게 광고되는 최대치는 213개.

그림 11. 도구 구성. 실측 레지스트리를 직접 불러 세었습니다(종료 코드 0). 중복 이름 0개.

확정 수치

238개 등록 · 205개 활성(레지스트리 `mode` 기준 읽기 53 / 쓰기 152) · 호환 전용 33개 · 세션 전용 8개를 더해 완전히 활성화된 쓰기 툴에서 모델에게 제공되는 최대치 213개.

조회

`get_map_region`

현재 상태를 읽습니다.

작성

`paint_tiles`

사본에 변경을 만듭니다.

관찰

`show_map_region`

현재 모습을 그림으로 봅니다.

검사

`run_lint`

데이터 오류를 검사합니다.

그림 12. 실제 도구 이름으로 구분한 역할 예시. 분야별 개수는 임의로 배분하지 않았습니다. 근거: [S030], [S031], [S033], [S035].

무슨 일을 하는 도구들인가

분야	대표 도구 (원문 이름)
맵 그리기	<code>paint_tiles</code> , <code>paint_road</code> , <code>fill_region</code> , <code>tile_erase</code> , <code>create_map</code> , <code>resize_map</code> , <code>generate_map</code> , <code>mirror_region</code> , <code>copy_map_region</code>
건축	<code>author_house</code> , <code>author_village</code> , <code>build_castle</code> , <code>build_wall</code> , <code>build_roof</code> , <code>place_door</code> , <code>place_window</code> , <code>lay_path</code> , <code>place_props</code>
실내·던전	<code>start_interior_room_session</code> , <code>furnish_interior_space</code> , <code>run_interior_room_pipeline</code> , <code>start_dungeon_room_session</code> , <code>evaluate_dungeon_room</code>
이벤트·사람	<code>upsert_event</code> , <code>place_npc</code> , <code>make_villager</code> , <code>set_npc_schedule</code> , <code>set_shop_stock</code> , <code>script_cutscene</code> , <code>place_chest</code> , <code>place_trap</code> , <code>author_npc_cast</code>
데이터베이스	<code>upsert_item</code> , <code>upsert_enemy</code> , <code>upsert_actor</code> , <code>upsert_skill</code> , <code>upsert_equipment</code> , <code>upsert_state</code> , <code>define_monster_species</code> , <code>set_type_chart</code> , <code>set_title_screen</code>
퀘스트·서사	<code>create_quest</code> , <code>define_quest</code> , <code>verify_quest</code> , <code>generate_walkthrough</code> , <code>author_story_arc</code> , <code>define_ending</code>
전투	<code>simulate_battle</code> , <code>tune_enemy</code> , <code>author_boss_phases</code> , <code>set_action_combat</code> , <code>make_action_enemy</code>
살펴보기(읽기)	<code>get_project_summary</code> , <code>get_map_region</code> , <code>find_events</code> , <code>get_event</code> , <code>get_database_records</code> , <code>tile_query</code> , <code>run_lint</code> , <code>check_reachability</code>
눈으로 보기	<code>show_map_region</code> , <code>preview_house</code> , <code>look_at_houses</code> — 이 셋만 그림을 돌려줍니다
되돌리기	<code>revert_last_edit</code> , <code>list_edit_history</code>

도구 입력값의 공통 검사는 기본 항목만 확인한다

`validateArgs` `jsonSchema.ts:229` 는 최상위 필수 항목·자료형·열거값·문자열 길이·모르는 항목 금지와 배열 원소 자료형만 봅니다. 숫자의 최소·최대, 여러 형태 중 하나(`oneOf`), 중첩된 객체의 필수 항목은 검사하지 않습니다. 그 부분은 각 도구 `run` 안의 도메인 검증이 담당합니다. 또 스키마상 필수인 `reason` (이 도구를 왜 쓰는지)이 빠져도 기록만 남기고 실행은 계속합니다 `assistantSession.ts:4565-4567`.

'호환 전용 33개'의 정확한 뜻

`toOpenAiTools` 에서 숨겨질 뿐 실행이 막힌 것은 아닙니다. `runTool` `toolRunner.ts:70-81` 은 폐기 여부를 검사하지 않으므로 `getTool` 로 이름을 직접 지정하면 여전히 실행됩니다. 예: `plan_village`, `build_village`, `build_house`, `clear_region`, `set_tile_metadata`, `show_tiles`, `scatter_object` 등. 대체 도구는 `author_village`, `author_house`, `tile_erase`, `tile_query` 입니다.

예외 하나 — 캐릭터 외형 생성

`generate_character_appearance` `characterAppearanceTools.ts:19` 는 레지스트리에서 읽기 도구로 등록돼 있지만 `isWriteToolName` `assistantSession.ts:668-672` 이 쓰기로 취급해 질문 모드에서 막습니다. 게다가 이 도구는 스스로 무엇을 저장하지 않고 UI를 열어 후보 그림만 만들며, 실제 프로젝트 반영은 사용자가 고른 뒤에 일어납니다

`characterAppearanceGeneration.ts:189` → 컨트롤러 `apply` :122.

9. 6단계 · 다른 심판이 검사한다

작업이 끝나면 바로 반영하지 않습니다. 작업 대화를 그대로 물려받지 않는 별도 검수 호출이 결과만 보고 승인 여부를 판정합니다. 이것이 이 편집기에서 사람 승인 버튼을 대신하는 장치입니다.

준비는 `reviewCurrentDraft` `assistantSession.ts:4008`, 요청 조립은 `buildIndependentReviewRequest` `independentReview.ts:176` 입니다.

심판에게 주는 것과 주지 않는 것

주는 것	주지 않는 것
사용자의 원래 요청문 그대로	작업자 모델의 설명·주장·대화 기록.
변경 전과 후의 실제 데이터 (<code>reviewChanges</code> :37)	"제가 잘 만들었습니다" 같은 자기 보고가 판정에 영향을 주지 못하게 일부러 제외합니다.
실제 도구 실행 결과	도구는 아예 주지 않습니다(<code>tools:[]</code> , <code>tool_choice:none</code>)
약속한 인수검사 조건	— 심판은 읽고 판정만 합니다.
첨부된 지도 그림	

증거가 넘치면 자르지 않고 거부한다

완전한 증거가 모델 창을 넘으면 증거를 잘라서 억지로 검수하지 않고 검수 자체를 거부합니다 `independentReview.ts:189-190`. 불완전한 정보로 승인이 나는 것을 막는 선택입니다.

모델이 "괜찮다"고 해도 통과하지 못하는 경우

`parseIndependentReview` :199 는 심판의 판정을 받은 뒤, 기계적으로 계산한 필수 문제 목록을 강제로 덧붙입니다 :223-226. 예를 들어 변경한 맵의 현재 모습을 `show_map_region` 으로 실제로 본 기록이 없으면 필수 문제로 남고 승인되지 않습니다 `assistantSession.ts:4074-4082`.

승인 후에도 `isDraftReviewApproved` :1250 가 다시 확인합니다 — 질문 모드가 아닌지, 기준 스냅샷이 최신인지, 검수가 취소되지 않았는지, 그리고 승인 당시의 내용과 지금 내용이 같은지. 하나라도 어긋나면 승인은 무효입니다.

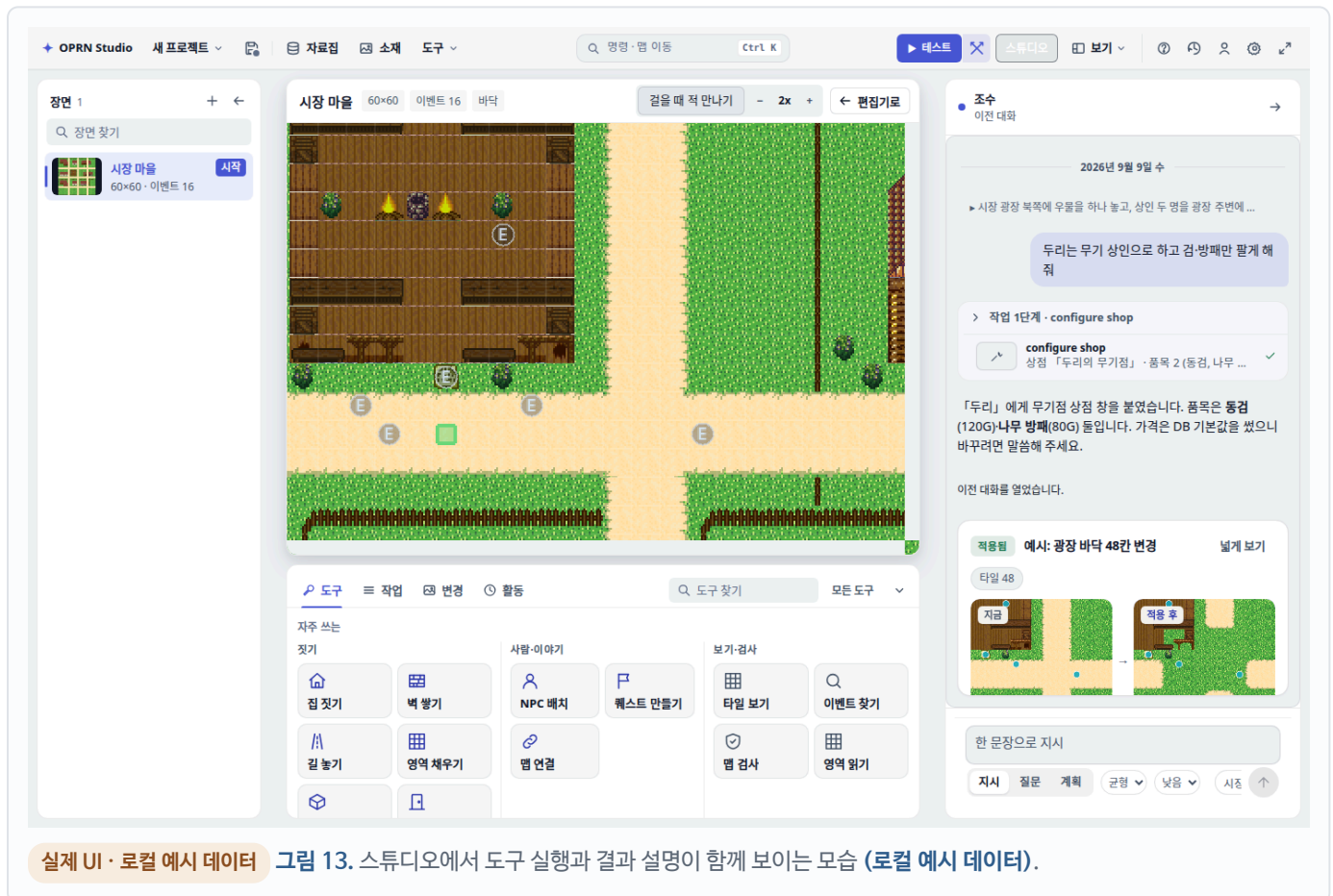
스스로 하는 인수검사 — "약속을 지켰는가"

검수와 별도로 요청 시점에 정한 약속을 지켰는지 확인하는 원장이 있습니다 (`AssistantAcceptanceLedger` `assistantAcceptanceLedger.ts:18`). `evaluate` :201 는 모든 증거가 통과하고 동시에 아직 반영 안 된 초안이 없을 때만 확인됨으로 판정합니다 :254-255. 계획을 건너뛰거나 다시 세워서 약속을 지우는 것은 불가능합니다.

확인 항목이 꽤 구체적입니다(`evaluateAcceptanceCriterion` `assistantAcceptanceEvaluation.ts:89`): 맵 개수·가로세로·이벤트 수가 정확히 맞는지, 바꾸라던 것은 바뀌고 지키라던 것은 지켜졌는지, 걸어서 갈 수 있는지, 그림 증거가 현재 내용을 덮는지.

기능 인수검사는 실제로 게임을 돌려 본다

`evaluateFunctionalCriterion` `functionalAcceptanceEvaluation.ts:28` 는 `runSceneTest` :74 로 실제 장면을 실행합니다. 물건 사기는 시작 지점 → 상인에게 접근 → 구매 → 소지금과 소지품이 실제로 변했는지까지, 왕복 이동은 나가는 이동과 돌아오는 이동 두 번 그리고 원위치 복귀까지 확인합니다. 그리고 모델은 테스트 스크립트나 "성공했다"는 표시를 넘길 수 없습니다 `functionalAcceptance.ts:45-54` — 모델은 기대하는 결과만 제시할 수 있고, 실제 성공 여부는 테스트 실행 장치가 판정합니다.



10. "승인"이라는 말의 세 가지 뜻

이 문서에서 가장 조심해서 읽어야 할 부분입니다. 같은 "승인"이라는 말이 서로 다른 세 가지를 가리키고 있어서, 셋을 합쳐 이해하면 "시가 물어보고 고친다"고 착각하게 됩니다. 일반 채팅에서는 물어보지 않습니다.

① 사람이 "실행해"라고 동의 — 있음

자율 수준을 "확인(confirm)"으로 두면

계획만 세우고 멈춘 뒤 "계속"을 기다립니다.

`autonomyLevels.ts:33 planOnly: true`

`finishPlanOnlyTurn`

`assistantSession.ts:2219 (runExecution=awaiting-user)`

· 실제로 작동하는 사람 게이트

단, 쓰기 하나하나를 묻는 게 아니라

"이 계획을 실행할까요?" 한 번입니다.

이미 있는 계획을 이어가는 턴은 그냥 실행됩니다.

② 화면에서 제안 카드 수락 — 일부 표면만

클러스터 모달과 타일셋 메타 모달

여기에는 실제로 누르는 버튼이 있습니다.

`clusterAiModal.ts:212-247` "수락해서 적용" / "거부"

`tilesetAiProposalModal.ts:915` "메타데이터 적용"

이벤트 편집기 보조도 체크박스로 고른 뒤

"[이대로 하기]"를 눌러야 반영됩니다.

`eventEditor/aiAssist.ts:379 runEventCommandAssist`

· 일반 채팅창에는 이 카드가 없습니다

→ 한 표면의 동작을 전체로 일반화하면 틀립니다.

③ 일반 채팅 쓰기 초안의 기계 검수

일반 채팅에서 실제로 막아 주는 것

별도 검수 호출이 승인해야 반영됩니다.

`isDraftReviewApproved`

`assistantSession.ts:1250`

`aiTurnRunner.ts:493-503` 3중 조건

`stoppedReason=final AND review=approved`

AND 세션 검수 신선험 — 하나만 어긋나도 적용 안 함

· 사람 대신 기계가 서는 문

여기에 걸리면 조용히 넘어가지 않고

"검수 미완료"로 표시됩니다.

헛갈리기 쉬운 것 — 이 둘은 "사람에게 묻는 문"이 아닙니다

· `reviewOverInsertion` (`overInsertionReview.ts:53`) — 파괴적 변경·타일 400칸 이상·이벤트 10개 이상을 감지하지만,

실제 사용처 (`aiProposalCard.ts:248-260`)는 이유만 표시하고 확인 없이 적용합니다. 복구 수단은 `Ctrl+Z`입니다.

· `proposalNeedsExplicitApproval` (`assistantSession.ts:377`) — 이름은 그럴듯하지만 실제 호출부가 없는 고아 함수입니다.

정의와 테스트 파일에서만 참조됩니다(`test/ruleApprovalGate.test.ts`). 살아 있는 승인 경로의 근거로 쓸 수 없습니다.

그림 14. 승인의 세 가지 뜻과, 승인처럼 보이지만 사람에게 묻지 않는 두 가지.

정리

무엇	사람이 누르나	어디에
계획 실행 동의	예 (한 번)	자율 수준을 '확인'으로 두었을 때. <code>planOnly:true</code> <code>autonomyLevels.ts:33</code> → <code>finishPlanOnlyTurn</code> <code>assistantSession.ts:2219</code> . 다음 메시지로 "계속"을 보내면 실행됩니다
제안 카드 수락	예	클러스터 모달 <code>clusterAiModal.ts:212-247</code> , 타일셋 메타 모달의 '메타데이터 적용' <code>tilesetAiProposalModal.ts:915</code> , 이벤트 편집기 보조의 '[이대로 하기]'
기계 결과 검수	아니오	일반 채팅. <code>aiTurnRunner.ts:493-503</code> 세 조건 + <code>applyProposedProject</code> 무결성 검사

승인처럼 보이지만 아닌 것 둘 — 실제로 확인하고 확인한 사실

- 1) 과삽입 검토 — `reviewOverInsertion` `overInsertionReview.ts:53` 는 파괴적 변경, 새 맵 생성, 쓰기 6건 이상, 타일 400칸 이상, 이벤트 10개 이상 추가를 감지합니다. 그런데 실제 사용처 `aiProposalCard.ts:248-260` 는 **이유만 표시하고 확인 없이 적용합니다**. 복구 수단은 Ctrl+Z입니다. 이 파일 상단 주석은 확인 모달처럼 설명하지만 현재 동작과 다릅니다.
- 2) `proposalNeedsExplicitApproval` `assistantSession.ts:377` — 이름은 "명시적 승인이 필요한가"이지만 실제 제품 실행 경로에서는 호출되지 않는 함수입니다. 정의와 테스트 파일(`test/ruleApprovalGate.test.ts`)에서만 참조됩니다. 살아 있는 승인 경로의 근거로 쓸 수 없습니다.

11. 7단계 · 실제 게임에 반영한다

검수를 통과하면 사본의 내용을 실제 프로젝트로 옮깁니다. 담당은 `applyProposedProject` `applyChangesetToStore.ts:240` 입니다.

반영 전에 서는 세 개의 문

1 기준이 낡지 않았는지

작업을 시작할 때의 프로젝트와 지금 프로젝트가 같은지 확인합니다 :245-247. 그 사이 사람이 직접 편집했다면 여기서 멈춥니다.

2 집을 망가뜨리지 않는지

`assertHouseProtection` :265-269. 이미 지어진 집 구조를 깨는 변경을 막습니다.

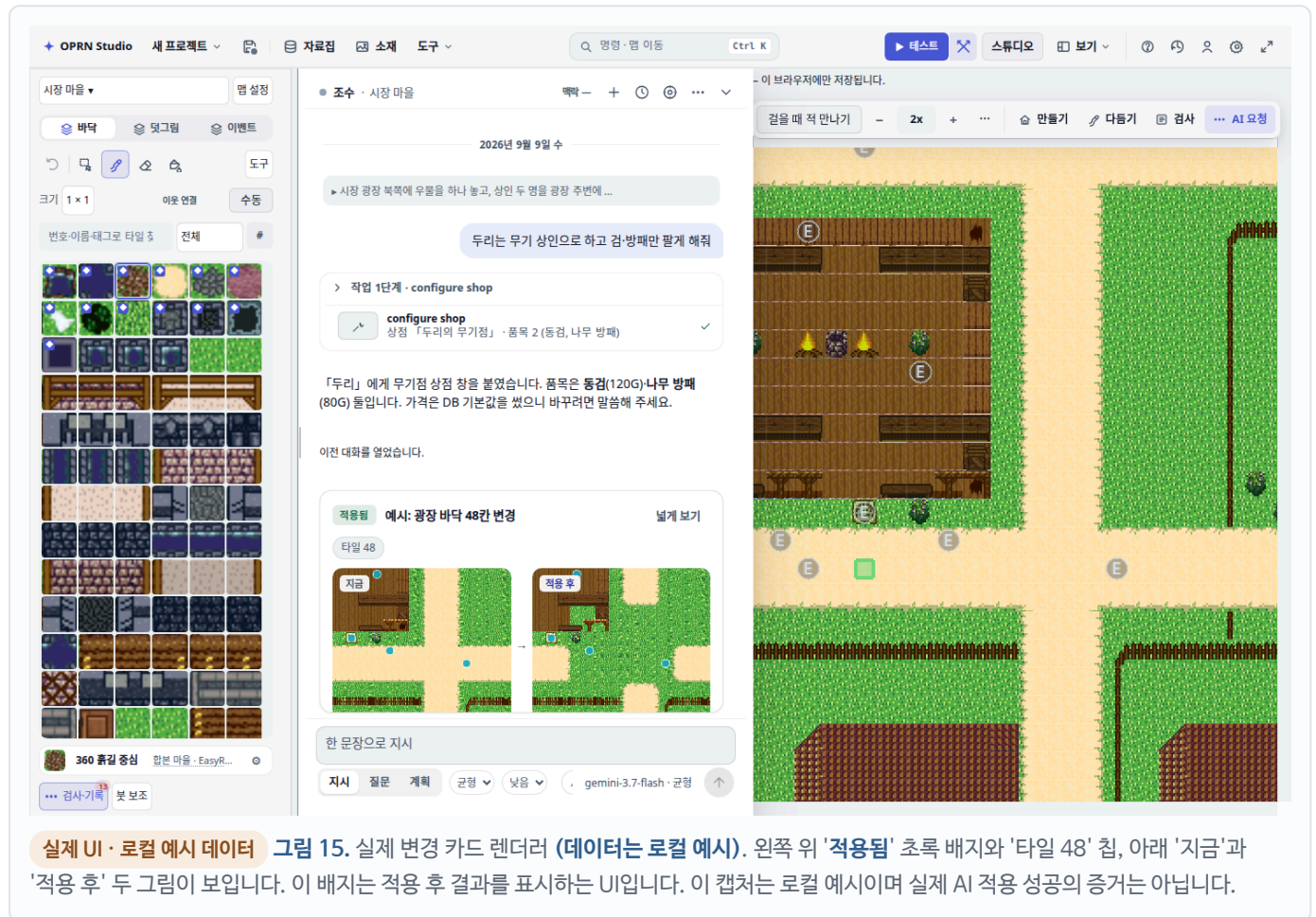
3 검사에서 새 오류가 생기지 않았는지

`commitChangeset` `changeset.ts:225`. 원래도 있던 오류는 통과시키고 새로 생긴 차단성 오류만 거부합니다 :227-236 — 기존 문제 때문에 새 작업이 전부 막히는 일을 피하려는 선택입니다.

세 문을 통과하면 `recordProjectSnapshot` `mapEditHistory.ts:175` 으로 되돌리기 지점을 남기고 `store.replace` `store.ts:805` 로 실제 반영, 그 다음 `recordProjectCommit` `projectCommitLog.ts:79` 로 변경 기록을 남깁니다.

반영 뒤에 보여 주는 것 — "변경 미리보기"는 사실 결과 카드

이름은 미리보기지만 이미 적용된 뒤에 나타나는 결과 카드입니다. `renderChangePreviewCard` `aiChangePreview.ts:143` 가 '적용됨' 배지 :159-160 와 변경 개수 칩, 그리고 '지금 / 적용 후' 두 장의 그림을 320픽셀로 보여 줍니다.



되돌리기의 두 가지 안전 수준

카드의 되돌리기와 컴포저의 되돌리기는 안전성이 다릅니다

컴포저의 되돌리기는 `completionUndoIsCurrent` `aiApplyCompletion.ts:67` 로 "지금 되돌릴 것이 정말 그 AI 작업인지" 확인한 뒤 동작합니다. 반면 대화 속 변경 카드의 되돌리기 버튼은 어느 카드인지 확인하지 않고 전역 `undoMapEdit` 을 바로 부릅니다 `aiChatPanel.ts:661-666`. 따라서 AI 작업 후 사람이 직접 편집한 상태에서 오래된 카드의 되돌리기를 누르면 사람이 한 편집이 되돌려질 수 있습니다(코드 근거이며 실제 화면 재현은 하지 않았습니다).

공통 경로를 지나지 않는 예외 하나

타일셋 AI 메타데이터 적용은 위 과정을 지나지 않습니다. '메타데이터 적용' 버튼 → `applyAnswerToTiles` `tilesetAiQuestionEditor.ts:181` → `store.update` :182 → `applyAiMappingAnswer` `tilesetAiMappingParser.ts:62` 로 바로 갑니다. 스냅샷 기록도, 독립 검수도 없습니다. 즉 이 경로에는 "적용 한 건당 되돌리기 지점"이 없습니다.

단, 복구 수단이 아예 없는 건 아닙니다

데이터베이스 모달 전체를 열었을 때 상태로 되돌리는 기능이 있습니다 — '열 때 상태로 되돌리고 닫기' `databaseModal.ts:548`, `createDatabaseModalDirtySession` :144. 또 저장 시점마다 커밋 기록이 남습니다 `projectCommitLog.ts:127-152`. 다만 모달의 되돌리기는 모달을 열었던 시점을 기준으로 하고, 커밋 기록은 저장 시점을 기준으로 합니다. 둘 다 AI 적용 한 건마다 남기는 되돌리기 지점과는 다릅니다. 그리고 다른 타일셋 편집 경로는 정상적으로 스냅샷을 기록합니다 `tilesetKnowledgeWorkspaceState.ts:240` — 이 한 패널만의 누락입니다.

12. 8단계 · 저장됐는지 따로 증명한다

가장 놓치기 쉬운 구분입니다. "적용됨"과 "저장됨"은 다른 상태입니다.

`store.replace` 는 화면상의 프로젝트를 바꾸고 자동 저장을 예약할 뿐입니다 `store.ts:830`. 실제 서버 저장은 `persistCurrent` :1243 가 `saveProjectMapPatchToSupabase` 또는 `saveProjectToSupabase` :1274-1276 를 부를 때 일어납니다. 그리고 변경 기록 저장이 네트워크 문제로 실패해도 적용 자체는 성공으로 보고됩니다 (`ok:true`, `persisted:false` `applyChangesetToStore.ts:311-319`).

진짜 저장 증명

`proveAppliedRevision` `assistantSession.ts:3249` 만이 확정 판정을 합니다: 대기 중인 저장을 즉시 실행하고 → 영수증을 받고 → `verifyPersistedRevision` `store.ts:917` 로 서버에서 프로젝트를 다시 읽어와 내용 해시와 세대가 맞는지 확인한 뒤해야 `verified=true` 가 됩니다. 원격 저장이 꺼진 환경(예: 개발용 쇼케이스)에서는 이 증명이 실패하고, 자동 되돌리기는 하지 않습니다.

결과를 세 축으로 따로 본다

`deriveRunOutcome` `runOutcome.ts:33` 은 결과를 실행(execution) / 목표(goal) / 전달(delivery) 세 개의 독립 축으로 유지합니다. 그래서 이런 조합이 가능합니다 — "모델이 정상적으로 응답을 마쳤다(실행 성공)"이지만 "사용자 목표는 아직 미달"이고 "서버 저장은 미확인". `stoppedReason=final` 은 목표 달성의 동의어가 아닙니다.

사용자에게 보여 주는 실행 요약

`buildRunRecap` `runRecap.ts:108` 은 모델별 사용량, 도구 성공·실패, 이어가기 횟수, 쓰기 건수, 중단 이유까지 저장하지만 채팅에 표시되는 것은 `formatRunRecapPlayerLine` :135 의 입력 토큰 / 출력 토큰 / 걸린 시간 한 줄뿐입니다 `aiTurnRunner.ts:426-431`. 상세 내역은 감사 기록과 활동 로그에 남습니다.

13. 분량 제한 — 네 겹의 서로 다른 예산

"AI가 기억할 수 있는 분량"이라고 뭉뚱그리기 쉽지만 실제로는 단위와 목적이 다른 제한이 네 개 겹쳐 있습니다.

A · 설명서 기본 100,000자 문자 예산	B · 작업창 최대 216,384토큰 모델 창보다 크지 않게
C · 전송 사본 오래된 결과·이미지 축소 원본 대화는 유지	D · 최종 요청 모델 창 기준 검사 예약 16,384토큰 포함

그림 18. 서로 다른 네 가지 분량 제한. 문자와 토큰을 같은 축에서 비교하지 않았습니다. 근거: [S003] ~ [S007].

겹	무엇	내용
A	시스템 설명서 문자 예산	<code>DEFAULT_BUDGET_CHARS</code> = 100,000자 <code>contextBuilder.ts:108</code> . 실제 응답의 문자/토큰 비율을 관찰해 보정하므로(<code>charsPerTokenEstimate</code> <code>tokenBudget.ts:50</code>) 실효 범위는 50,000~150,000자. 넘치면 단순 절단
B	자동 압축 작업창	<code>min(모델 창, 216384)</code> <code>messageBudget.ts:88</code> . 작업창이 216,384토큰일 때 압축 시작 문턱은 200,000 토큰 <code>contextCompaction.ts:70</code> . 실측 <code>shouldCompact(200000, 216384)</code> =거짓, <code>200001</code> =참
C	보내는 사본 깎기	<code>compactMessagesForRequest</code> <code>messageBudget.ts:151</code> . 세션 원본은 건드리지 않고 보낼 사본에서만 오래된 도구 결과와 이미지를 덜어냅니다. 절대 상한 보장은 아님
D	실제 요청 최종 검사	<code>buildGroundedRequest</code> <code>contextBuilder.ts:663</code> . 도구 스키마 분량 + 예약 16,384 + 원본 목록까지 계산. 넘치면 도구를 잘라 통과시키지 않고 <code>original-context-window-exceeded</code> 오류 :685-687

압축은 무엇을 남기고 무엇을 버리는가

`runCompaction` assistantSession.ts:3541 이 돌면: 남는 것은 시스템 설명서, 새로 만든 요약 한 덩어리(`[context-compaction-summary]` 표시가 붙음), 그리고 최근 대화 꼬리입니다. 버리는 것은 그 앞의 원래 대화입니다. 요약을 만들 때 이미지는 제거하고 도구 결과는 건당 2,000자로 줄입니다 contextCompaction.ts:359-401. 사용자 한 턴에 자동 압축은 최대 1회이고(성공·실패 무관 :3546,:3595), `undoLastCompaction` :1205 으로 한 단계 되돌릴 수 있습니다.

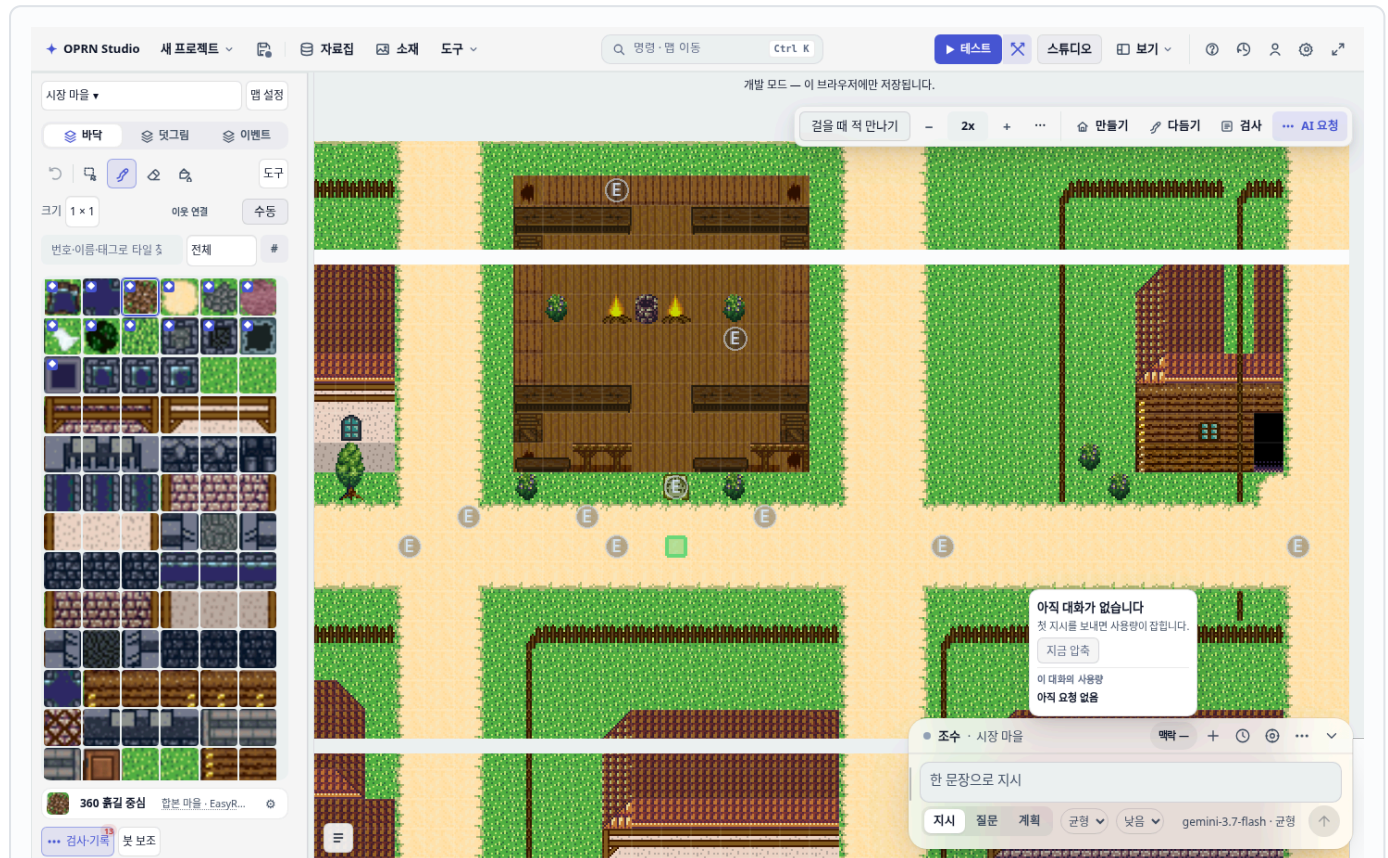


그림 19. 컴포저의 '맥락' 미터 실제 화면(`createAiContextMeter` aiContextMeter.ts:70). 현재 토큰, 작업창, 자동 압축 문턱, '지금 압축', '압축 되돌리기', 요약 원문 보기, 모델별 사용량을 보여 줍니다.

미터 숫자가 압축 후에도 안 줄어 보일 수 있습니다

미터가 쓰는 `lastPromptTokens` 는 모델 응답이 알려 준 과금 토큰으로만 갱신되고 assistantSession.ts:3518, 압축이나 되돌리기 때 초기화되지 않습니다. 그래서 직전에 큰 요청이 있었다면 압축 직후에도 그 값이 남아 게이지가 높게 보일 수 있습니다. 계산 실제로 `describeContextUsage` 에 과금 210,000 / 창 216,384를 주고 대화를 아주 짧게 두면 비율 0.97, 문턱 초과 = 참이 나옵니다(코드로 직접 확인).

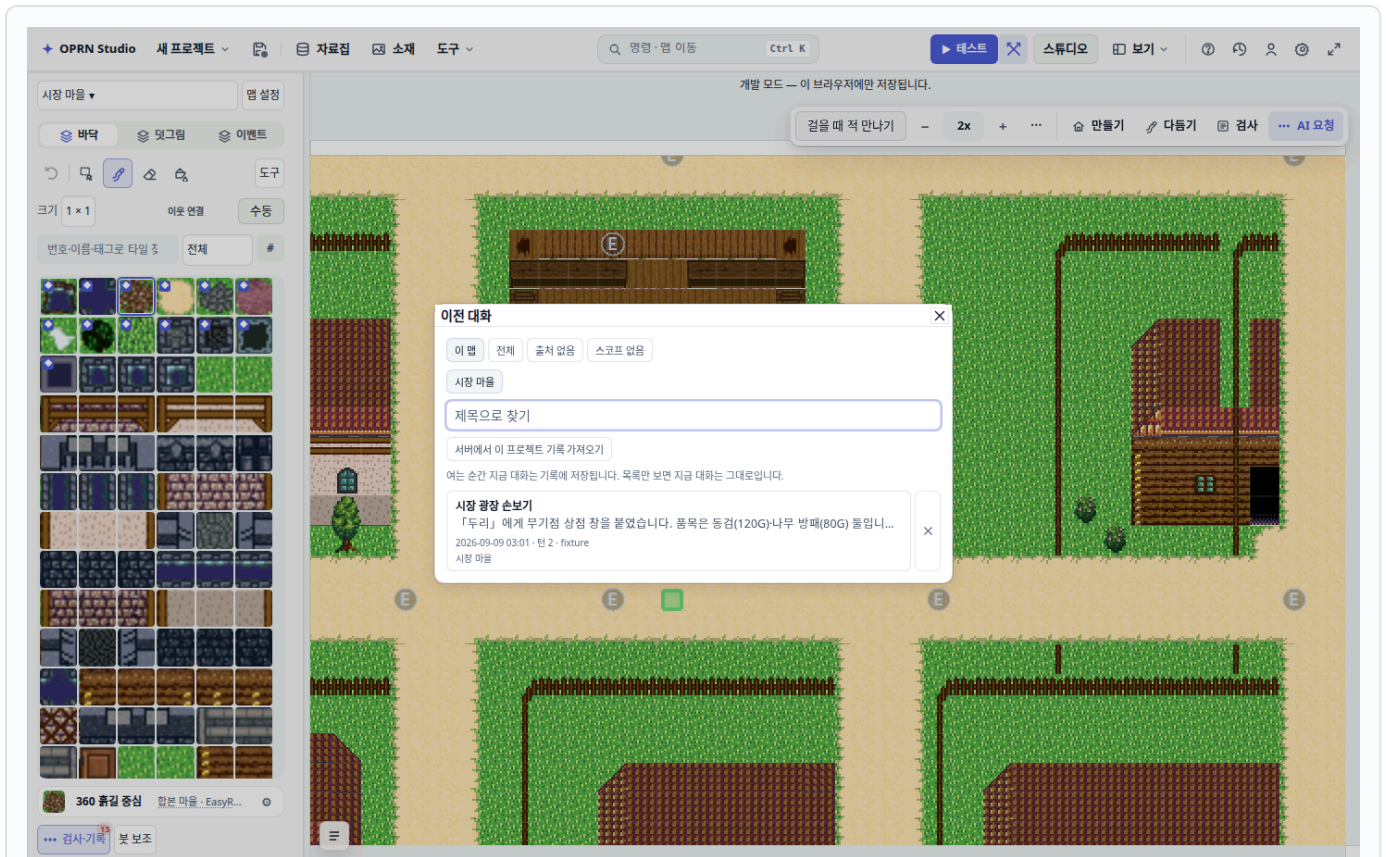
14. AI는 무엇을 기억하는가

세 가지가 서로 다른 곳에 서로 다른 방식으로 남습니다.

무엇	어디에	특징
대화	브라우저 IndexedDB <code>oprn-ai-records</code> / <code>conversations</code> <code>aiRecordDb.ts:13,15</code> + 서버 <code>rpg_zzu.ai_conversations</code> 의 <code>entries_json</code>	정본은 브라우저, 서버는 미러. 옛 <code>localStorage</code> <code>oprn:ai-conversations</code> 는 첫 접근에 옮기고 지웁니다 <code>conversationStore.ts:228</code>
활동 기록	<code>localStorage</code> 100건 링버퍼 <code>activityLog.ts:21-22</code> + <code>rpg_zzu.ai_activity_logs</code> + 디스크 <code>output/ai-activity/</code>	한 턴에 무슨 도구를 어떤 인수로 불러 성공했는지, 감사 항목, UI 조작까지. 예산 초과 시 앞뒤를 남기고 가운데를 줄입니다
사람 성향	<code>localStorage</code> <code>oprn:ai-preference-memory</code> <code>preferenceMemory.ts:41</code>	서버 미러 없음. 이 브라우저에만 남습니다

되돌리기 기록이 이후 대화에 반영될 수 있습니다

성향 기억은 사용자가 명시적으로 적어 주지 않아도 쌓입니다. `observeTurn` `preferenceSignals.ts:375` 와 `noteAiChangeUndone` :382 가 신호를 모으는데 가중치가 이렇습니다 — 되돌리기 -3, 정정 -2, 그대로 두면 +1. 신호가 쌓이면 (`shouldDistillPreferences` :276, 보류 6건 이상 또는 사용자가 직접 말한 경우 즉시) 설정된 실행 모델이 한 문장짜리 사실로 정리하고 (`distillPreferences` :175, 한 번에 최대 3건) 그 결과가 다음부터 시스템 설명서에 실립니다 (`buildPreferenceMemorySection` `preferenceMemory.ts:358` → 일반 채팅의 `systemPromptOptions` `assistantSession.ts:1158`). 별도 보조 화면 일부는 `composeSystemPrompt` 를 사용합니다. 되돌리기 한 번이 곧바로 새 기억을 만드는 것은 아닙니다. 분량은 전역 16건 / 프로젝트 8건, 블록 12줄·1,200자로 제한되고 `PREFERENCE_PRECEDENCE_LINE` :62 이 "사용자의 지시가 이 기억보다 우선"이라고 명시합니다.



실제 UI · 로컬 예시 데이터 그림 20. 과거 대화 목록 실제 화면(`openAiConversationHistoryModal`

`aiConversationHistoryModal.ts:130`). 맵별 필터로 찾을 수 있습니다.

과거 대화를 이어갈 때

화면 복원과 모델에게 주는 것은 다른 경로입니다. 모델에게는 `serializeAuditTranscript` `conversationReplay.ts:40` 로 정리한 기록을 `restoredTranscriptMessage` :78 가 사용자 역할의 한 덩어리로 넣고, `RESTORED_TRANSCRIPT_MARKER` 와 함께 "이건 지난 기록이니 다시 실행하지 마라"고 명시합니다.

15. 설정한 값이 그대로 가지 않는 구간

설정 화면에서 고른 값 중 일부가 실제로 모델에게 전달되지 않습니다. 이 구간을 모르면 "추론 강도를 높였는데 왜 변화가 없지?" 같은 의문이 풀리지 않습니다.

브라우저는 보낸다

`chatCompletion` `llmClient.ts:791` 이 만드는 요청 본문에는 `max_tokens`, `temperature`, `reasoning` (추론 강도), `tool_choice`, `response_format`, `stream` 이 모두 들어갑니다 :521-555.

그런데 중간 어댑터가 버린다

확정된 손실 경계 — 필드별로 전부 확인했습니다

서버 쪽 `completeProvider` `scripts/lib/ohMyPiAiRuntime.ts:208-242` 는 `openaiToContext` 로 메시지와 도구 목록만 변환하고, 실제 공급자 호출에는 `apiKey` 와 `fetch` 만 넘깁니다 :238-241.

따라서 `max_tokens` · `temperature` · `reasoning` · `tool_choice` · `response_format` · `stream` 은 전달되지 않습니다. 반환은 항상 비스트리밍 JSON이고 :242, `assistantToOpenAI` :157-205 는 사용량(usage)과 추론 내용도 돌려주지 않습니다. 이 어댑터는 죽은 코드가 아닙니다 — `oh-my-pi-worker.ts:22-33` 의 라이브 경로가 실제로 부릅니다.

정확한 해석: "호출자가 지정한 값이 버려진다"는 뜻이며, "공급자가 자체 기본값이나 추론을 못 쓴다"는 뜻은 아닙니다. 또 세션이 자체적으로 강제하는 예산(라운드 상한 등)은 별개 계층으로 여전히 작동합니다.

연결과 모델

기본 모델은 `gemini-3.7-flash` `llmClient.ts:84,86` 이고, 현재 UI가 노출하는 연결은 두 가지 구독 로그인(OAuth)뿐입니다 — Antigravity(기본)와 Codex `ohMyPiProviders.ts:38-53`. 설정 화면에 'API 키'라는 라벨의 선택 항목은 남아 있지만 키를 입력하는 칸을 만들지 않으며 `aiAuthSettings.ts:232-234`, 힌트에 "아래 두 제공자는 모두 구독 로그인입니다"라고 적혀 있습니다 :68-70. 어느 쪽을 골라도 같은 두 제공자가 나오고 내부적으로 구독 방식으로 강제됩니다 `aiConnectionKind.ts:76-83`.

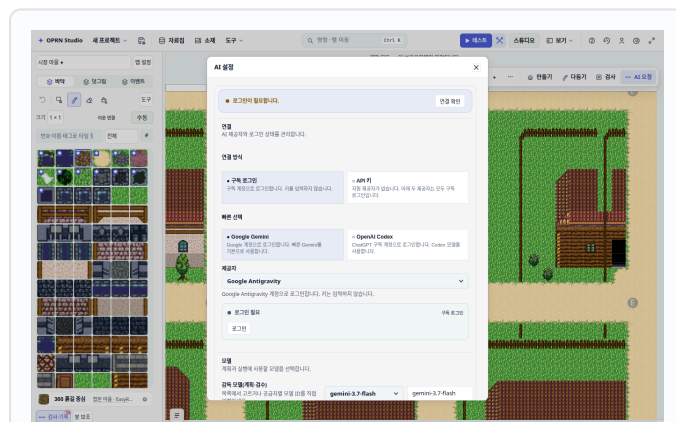


그림 21. AI 설정 모달 — 감독 모델과 실행 모델을 따로 고릅니다.



그림 22. 연결 설정 확대 — 키 입력칸이 없고 로그인 방식만 있습니다.

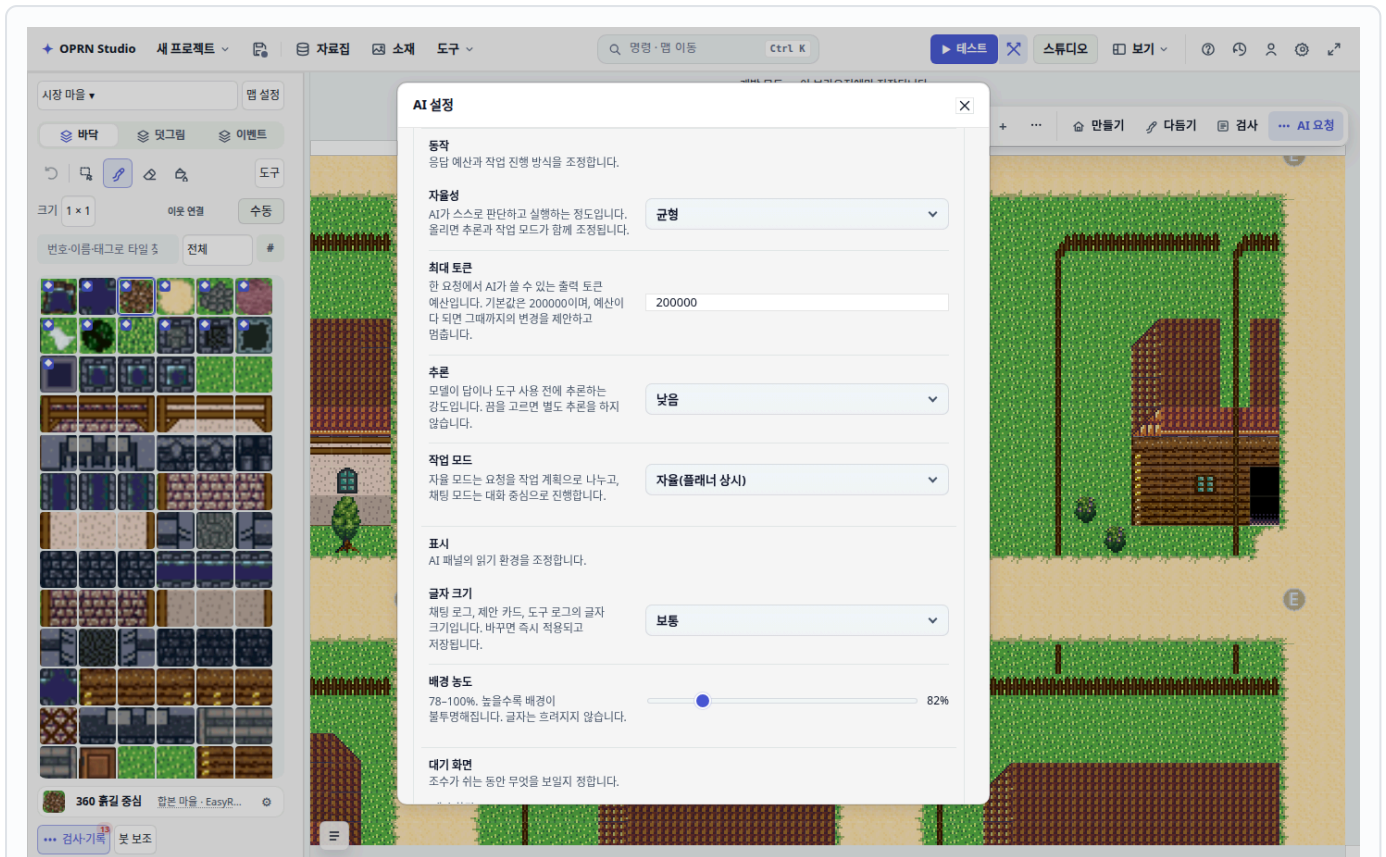


그림 23. 행동 설정 — 자율 수준과 추론 강도. 다만 추론 강도는 위에서 설명한 손실 경계 때문에 현재 companion 경로에서 공급자에게 전달되지 않습니다.

부수적으로 확인된 것 — 저장된 자격 증명은 지워지지 않습니다

`loadAiConfig` `llmClient.ts:192` 는 돌려주는 설정값에서 키와 주소를 비워 구독 방식으로 교정하지만 `:199, :245, :249,` 저장된 JSON 자체는 덮어쓰지 않습니다(읽기만 합니다 `:196`). 스크립 함수 `scrubStoredAiCredentials` `:157` 는 존재하지만 테스트에서만 호출되고 실제 제품 실행 경로에서는 호출되지 않는 함수입니다. 따라서 "예전에 넣은 비밀이 부팅 때 물리적으로 지워졌다"고 말할 수 없습니다.

16. 실제 화면 모습

아래는 이 조사 중 실제 편집기를 띄워 캡처한 화면입니다 (`127.0.0.1:9841` , 1440×900, 페이지 오류 0건).

이 화면들의 성격 — 반드시 함께 읽어 주십시오

실제 모델 호출은 하지 않았습니다. 대화·작업 계획·변경 비교 화면은 로컬 예시 데이터를 실제 제품 렌더러에 표시한 것입니다. 즉 UI 구조와 배치는 진짜지만, "AI가 실제로 이렇게 성공했다"는 증거로 읽으면 안 됩니다.

또한 현재 기본 제공자(Antigravity)가 로그인 필요 상태이므로 연결 성공 화면, 맥락 사용량 수치가 채워진 화면, 타일셋 분석 결과 화면은 확보하지 못했습니다. 제품 원래 모습을 그대로 담았기 때문에 일부 모달 뒤 어두운 띠나 좁은 열의 어색한 줄바꿈이 보입니다 — 가리거나 수정하지 않았습니다.

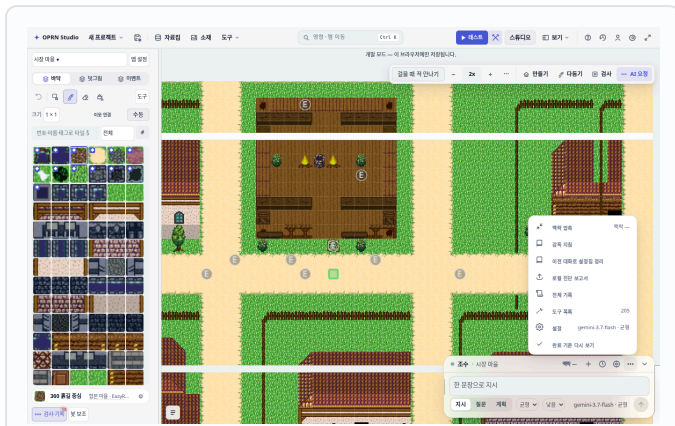


그림 24. 조수 명령 메뉴 — 대화 관리·설정 진입점.



실제 UI · 로컬 예시 데이터 그림 25. 대화 확대 — 말풍선과 도구 카드 배치 (로컬 예시).

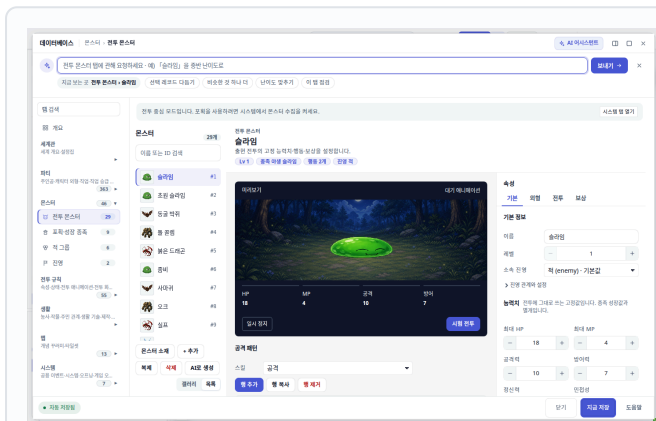


그림 26. 데이터베이스 시 — '지금 보는 곳' 맥락 칩과 추천 문구.

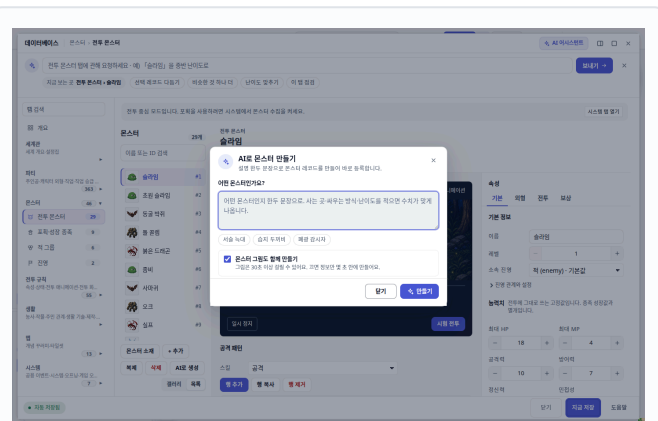


그림 27. 'AI로 몬스터 만들기' — 3층 직접 호출 경로.

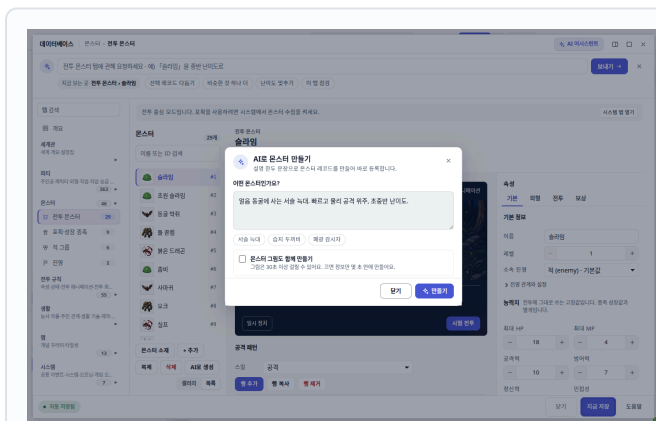


그림 28. 같은 대화상자 입력 후 — 만들면 곧바로 목록에 등록됩니다.

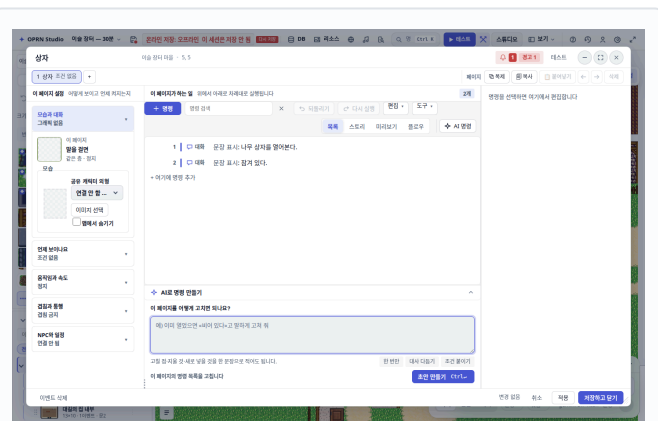
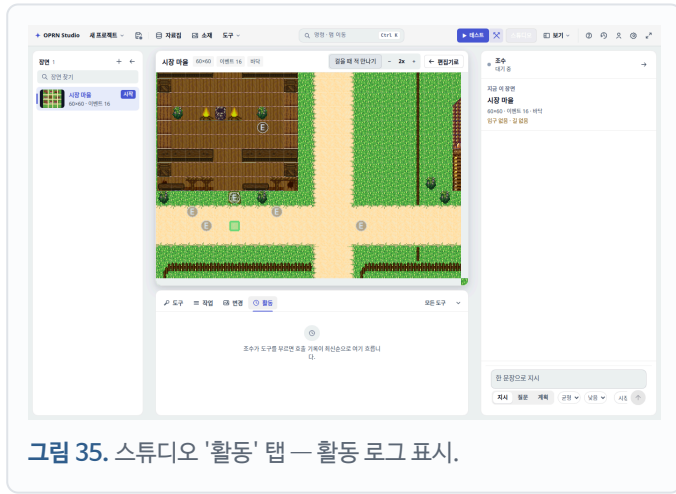
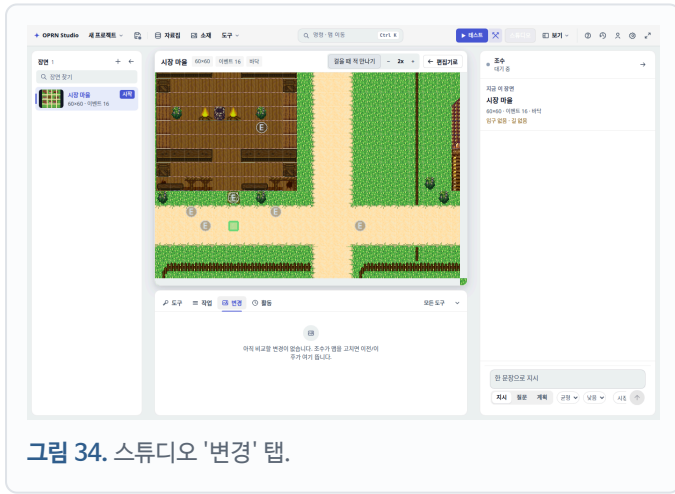
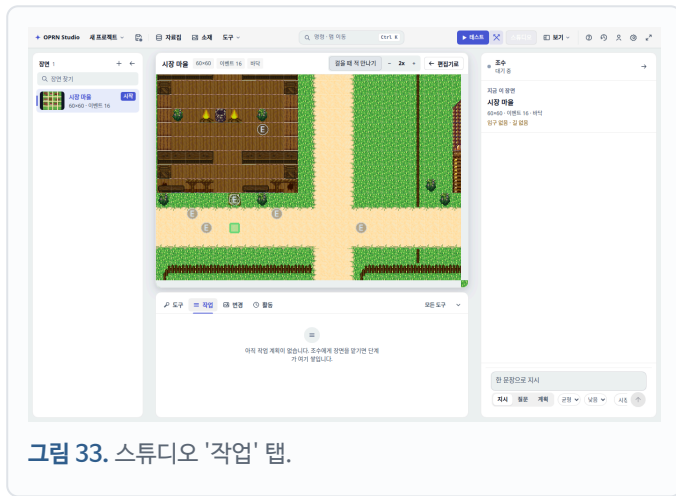
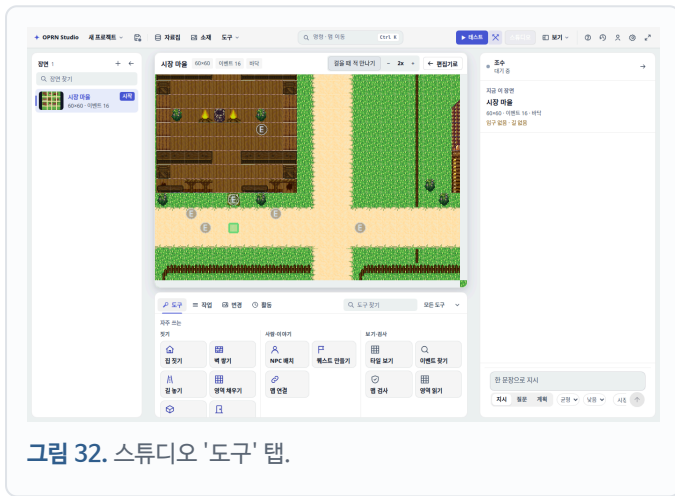
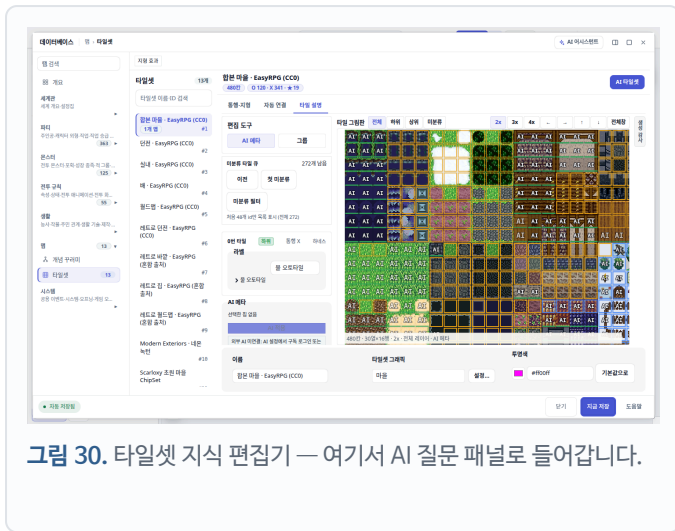
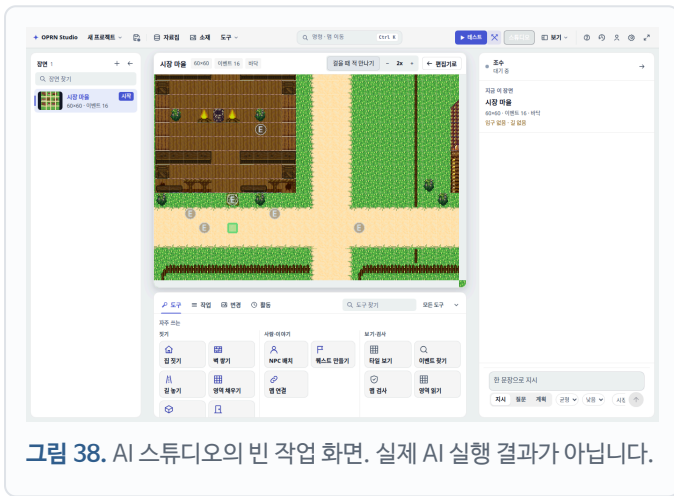
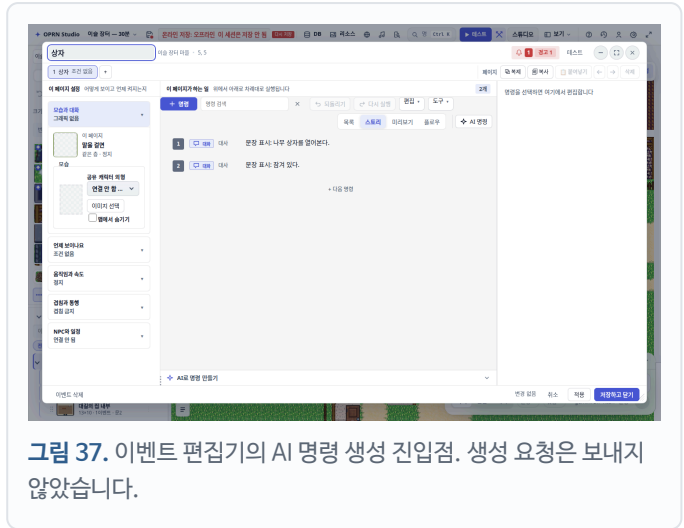
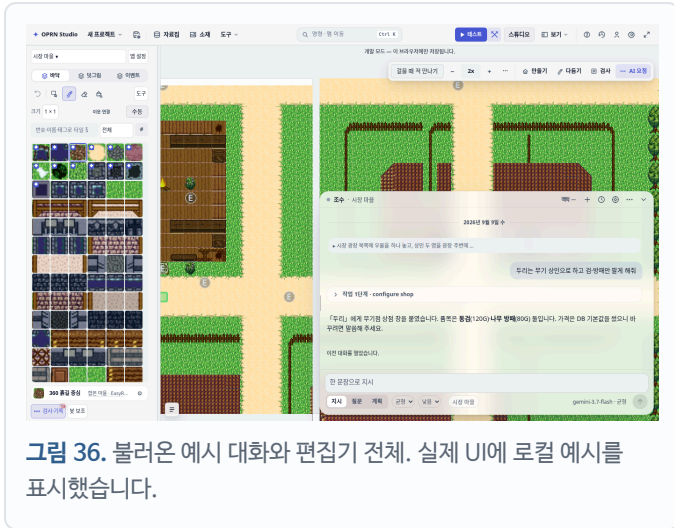


그림 29. 이벤트 편집기의 시 보조 독.





캡처하지 못한 화면 (코드 근거로만 서술한 부분)

연결 성공 상태 · 실제 AI 작업 진행 중 화면 · 맥락 사용량 수치가 채워진 미터 · 타일셋 분석 후 질문·결과·적용 화면 · 독립 검수가 반려한 '검수 미완료' 표시. 현재 기본 제공자가 로그인 필요 상태이고 이 조사에서 인증을 진행하지 않았기 때문입니다.

17. 부록

A. 함수 이름 색인 — 무슨 일을 하는가

이름 (원문)	위치	하는 일
<code>renderAiChatPanel</code>	<code>aiChatPanel.ts:290</code>	채팅 패널 전체를 그리고 배선. 채팅 패널을 조립하는 공개 함수
<code>send</code> / <code>sendText</code>	<code>aiChatPanel.ts:1677</code> / <code>:1527</code>	보내기 버튼·Enter의 공통 입구 / 일반 채팅 공통 통로 (둘 다 내부 함수)
<code>createAiTurnRunner</code>	<code>aiTurnRunner.ts:123</code>	한 턴의 실행·진행 표시·적용 게이트를 담당하는 실행기 생성

이름 (원문)	위치	하는 일
AssistantSession	assistantSession.ts:863	조수 세션 본체. 대화·사본·도구·검수·적용 조정 (5,319줄)
sendUserMessage	assistantSession.ts:1686	세션의 유일한 사용자 메시지 입구
executeUserTurn	assistantSession.ts:1874	한 턴의 준비: 맥락 포착, 의도 선언, 필요하면 계획
runTurnLoop / executeTurnLoop	:4209 / :4230	검수 권한 수명 관리 / 실제 생각-도구 반복
runAutonomousDriver	assistantSession.ts:1785	턴이 끝난 뒤 "계속"을 스스로 보내는 자동 진행 (최대 48턴)
createLlmIntentDeclarer	intentDeclarationClient.ts:89	설정된 실행 모델로 요청의 뜻을 JSON으로 분류
runOrchestratorPlanner	assistantSession.ts:2300	큰 요청을 총·항목 작업 계획으로 분해
buildSystemPrompt	contextBuilder.ts:693	지금 프로젝트 상황 설명서 조립
buildGroundedRequest	contextBuilder.ts:663	실제 요청 조립 + 모델 창 최종 검사
extractOriginalContext	originalContext.ts:117	편집 전 원본 묶음(이벤트 전체 페이지 포함) 추출
chatCompletion	llmClient.ts:791	모델 호출 단일 통로
TOOL_REGISTRY / activeTools / toOpenAiTools	toolRegistry.ts:156 / :234 / :266	도구 정보 238개 / 활성 205개 / 모델용 변환
runTool / runToolDefinition	toolRunner.ts:70 / :84	이름으로 도구를 찾아 실행 / 검증·실행·변경 격리
cloneDetachedDraft	detachedDraftMemory.ts:25	실제 프로젝트와 분리된 작업 사본 생성
reviewCurrentDraft / buildIndependentReviewRequest	assistantSession.ts:4008 / independentReview.ts:176	독립 검수 실행 / 검수 요청 조립 (작업자 발언 제외)
isDraftReviewApproved	assistantSession.ts:1250	지금 이 초안이 정말 승인된 상태인지 재확인
applyProposedProject	applyChangesetToStore.ts:240	세 개의 문을 지나 실제 프로젝트에 반영
renderChangePreviewCard	aiChangePreview.ts:143	적용 후 전/후 비교 카드 표시
proveAppliedRevision	assistantSession.ts:3249	서버에서 다시 읽어와 저장 사실을 증명
deriveRunOutcome	runOutcome.ts:33	실행·목표·전달 세 축을 독립적으로 판정
maybeCompactConversation / compactMessagesForRequest	assistantSession.ts:3533 / messageBudget.ts:151	대화 자동 압축 / 보내는 사본만 깎기
AssistantAcceptanceLedger	assistantAcceptanceLedger.ts:18	요청 시점 약속을 계획과 별도로 보관·판정
buildPreferenceMemorySection	preferenceMemory.ts:358	학습한 사람 성향을 설명서에 실음

B. 프로젝트 문서와 코드가 어긋난 곳

조사 중 프로젝트 자체 위키와 코드 주석에서 현재 코드와 다른 서술을 다수 확인했습니다. 이 문서는 모두 코드를 정본으로 판정했습니다.

문서·주석의 서술	실제 코드
도구 노출 상한 40개 (MAX_EXPOSED_TOOLS), 활성화 148개 openwiki/editor-ai-tools.md:557,577,606,616	해당 심볼이 존재하지 않고 assistantSession.ts:4271 은 무제한 toOpenAiTools() 사용. 실측 활성화 205개
시스템 설명서 예산 12,000자, 요청 사본 52,000자 고정 openwiki/ai-context-compaction.md	100,000자 + 관찰 보정(50k~150k) contextBuilder.ts:108
대화는 localStorage만 사용 openwiki/ai-workflow.md	IndexedDB oprn-ai-records 정본, localStorage는 이관용 conversationStore.ts:31,45,224
영역 작업 도구 호출 상한 24회 openwiki/editor-ai-tools.md	2000회 runRegionTask.ts:70, assistantEndpoint.ts:62
오류가 난 턴의 성공 쓰기도 적용된다 approvalPolicy.ts:34-35 주석	aiTurnRunner.ts:493-503 이 stoppedReason=final 이 아니면 적용 전에 반환
환경 변수나 localStorage의 API 키를 풀백으로 쓴다 llmClient.ts:66-67 주석	현재 로더는 구독 방식으로 강제 :101-105,:199
과삽입 검토가 사용자 확인을 요구한다 overInsertionReview.ts 상단	실제 사용처는 비차단 경고 aiProposalCard.ts:248-260
find_tools 설명에 "현재 40개 작업 툴" discoveryTools.ts:43	전체 활성화 스키마가 첫 요청부터 노출됨
present_doc 설명이 table·html 지원을 약속 aiDocTools.ts:126-130	모델에게 주는 스키마의 종류 열거에는 그 둘이 없음 :143

C. 이 조사에서 확정하지 못한 것

결함으로 단정하지 않은 두 가지 — 정적 근거만 있고 재현하지 않았습니다

1) 응답 본문이 멈췄을 때 중단이 전달되는지 — llmClient.ts:934-950 은 응답 헤더를 받은 직후 시간 제한과 중단 감지를 해제하는데, 그 뒤 본문을 읽는 parseSseStream :638 의 읽기 루프는 중단 신호를 구독하지 않습니다 :684. 따라서 본문이 멈추면 중단 요청이 읽기를 깨우지 못할 가능성이 있습니다. 다만 현재 companion 경로는 비스트리밍이므로 이 경로의 실제 영향은 분리해서 봐야 합니다.

2) 도구 묶음 실행 중 예상 밖 예외가 났을 때 — assistantSession.ts:4801-4812 는 현재 호출에는 실패 응답을 남기고 예외를 다시 던지는데, 같은 응답에 뒤따르는 다른 도구 호출에 대한 응답 짝을 맞추는 정리가 보이지 않습니다.

그 밖에 이 조사가 하지 않은 것: 실제 모델 공급자 호출, 실제 토큰 소비 측정, OAuth 외부 로그인 성공 확인, author_village 내부 단계의 상세 순서와 실패 재시도 분기, 위 두 후보의 결정론적 재현.

D. 조사 방법

항목	내용
기준 소스	HEAD <code>c56628804</code> , 조사 중 <code>72f1f179</code> 로 이동했으나 <code>git diff --name-status c56628804 HEAD -- src</code> 결과가 비어 있어 인용된 모든 코드는 두 시점에서 동일
병렬 조사	축별 담당 8명(턴 루프 / 맥락 / 모델 계층 / 도구 / 적용 / 계획·검사 / 기억 / 반론) + 레인 5(위키 대조, 표면 지도, 심볼 색인, 화면 캡처, 예제 추적)
반론 라운드	6회. 함수 이름이 실제로 존재하는지, 인용한 경로가 죽은 코드가 아닌지를 전담 검증. 결과: 정정 2건 (PARTIAL), 확정 4건 (CONFIRMED)
실행 검증	도구 레지스트리 직접 import (종료 코드 0) → 238/205/53/152/33, 중복 0 · 압축 판정 함수 직접 호출 · 편집기 실제 화면 31장 캡처
이름 확인 원칙	이 문서에 적힌 모든 함수·도구·상수 이름은 소스에서 직접 확인한 것만 사용. 존재하지 않는 이름을 쓰지 않기 위해 반론 담당이 별도로 재확인
작성 시점	최초 조사 2026-09-09(KST). 이어진 작업에서 문장·출처·캡처 표기를 수정했습니다. 최종 경과 시간은 전달 기록에 남깁니다.

이 문서를 다시 쓸 때 기억할 것

프로젝트 위키의 AI 관련 쪽에는 오래된 숫자가 상당히 남아 있습니다(부록 B). 수치를 인용할 때는 반드시 코드에서 다시 확인하시고, 특히 도구 개수와 예산 상수는 레지스트리와 상수 선언을 직접 보는 것이 가장 빠릅니다.

출처와 확인 범위

출처 원장 64항목입니다. 일부는 여러 파일을 묶은 항목이고, 마지막 세 항목은 실행·캡처 증거입니다. 웹 검색 출처는 없으며 외부 웹 도메인 수는 0입니다. 코드 해석은 조사 당시 스냅샷에 한정합니다.

- [S001] `src/ai/assistantSession.ts` (5,319줄) — 턴 루프·검수·적용 조정의 중심
- [S002] `src/ai/llmClient.ts` (984줄) — 모델 호출·설정·재시도·SSE 파싱
- [S003] `src/ai/contextBuilder.ts` (798줄) — 시스템 프롬프트 조립, `buildGroundedRequest`
- [S004] `src/ai/contextCompaction.ts` (511줄) — 압축 판정·요약 요청
- [S005] `src/ai/messageBudget.ts` (271줄) — 작업창·전송 사본 클램프
- [S006] `src/ai/tokenBudget.ts` — 문자/토큰 비율 보정
- [S007] `src/ai/originalContext.ts` — 원본 스냅샷·페이징
- [S008] `src/ai/workPlan.ts` (1,028줄) — 계획 분해·Ralph 계속
- [S009] `src/ai/intentDeclaration.ts` / `intentDeclarationClient.ts` — 의도 분류
- [S010] `src/ai/independentReview.ts` — 독립 검수 요청·파싱
- [S011] `src/ai/assistantAcceptanceLedger.ts` / `assistantAcceptanceEvaluation.ts` / `assistantAcceptance.ts` / `assistantAcceptanceTools.ts` — 인수검사
- [S012] `src/ai/functionalAcceptance.ts` / `functionalAcceptanceEvaluation.ts` — 기능 인수검사

[S013] src/ai/approvalPolicy.ts / autonomyLevels.ts / capabilityEscalation.ts — 적용 모드·자율 수준

[S014] src/ai/proposalCompleteness.ts / overInsertionReview.ts — 완성도·과삽입 검토

[S015] src/ai/runOutcome.ts / runRecap.ts / workItemOutcome.ts / volumeContract.ts — 결과 판정

[S016] src/ai/agentVerification.ts / toolReadEvidence.ts / toolVerificationEvidence.ts — 증거 관리

[S017] src/ai/conversationStore.ts / mapConversationStore.ts / conversationReplay.ts / aiRecordDb.ts — 대화 저장

[S018] src/ai/activityLog.ts / activityLogTypes.ts / activityLogEndpoint.ts / activityRunId.ts / uiEventLog.ts — 기록

[S019] src/ai/preferenceMemory.ts / preferenceSignals.ts / preferenceDistiller.ts — 선호 기억

[S020] src/ai/modelCatalog.ts / ohMyPiProviders.ts / aiConnectionKind.ts / imageModelCatalog.ts — 모델·제공자 목록

[S021] src/ai/oauth/credentials.ts / codexBrowserOAuth.ts / codexDeviceOAuth.ts / antigravityOAuth.ts / chatgptOAuthClient.ts — 인증

[S022] src/ai/imageGenerationClient.ts / imageGenerationQueue.ts / imageReferences.ts — 이미지 생성

[S023] src/ai/toolImageRenderer.ts / toolImageCanvas.ts / toolImageEventSprites.ts — 도구 이미지 렌더

[S024] src/ai/systemPromptEnvelope.ts / promptPolicies.ts / projectInstructions.ts / contextFooter.ts / turnGuide.ts — 프롬프트 부품

[S025] src/ai/projectWikiContext.ts / worldCanonContext.ts / villageDesignContext.ts / mapViewportContext.ts — 맥락 자료

[S026] src/ai/toolCapabilityIndex.ts / planToolExposure.ts / toolReason.ts — 도구 색인·이유

[S027] src/ai/buildSpec.ts (837줄) — 공간 명세 검증

[S028] src/ai/npcCast.ts / requestCoverage.ts / eventCommandAssist.ts / eventPageSemantics.ts — 저작 보조

[S029] src/ai/sessionUsage.ts / conversationTurnContext.ts / yieldToUi.ts / composerMode.ts / messageBudget.ts — 부족

[S030] src/editor/tools/toolRegistry.ts — 도구 정보 238개

[S031] src/editor/tools/toolRunner.ts / jsonSchema.ts / changeset.ts / types.ts — 실행·검증·격리

[S032] src/editor/tools/applyChangesetToStore.ts — 실제 적용

[S033] src/editor/tools/mapTools.ts / eventTools.ts / dbTools.ts / queryTools.ts — 주요 도구군

[S034] src/editor/tools/v3/constructionTools.ts / authorHouseToolDef.ts / authorVillageToolDef.ts / villageTerrainPass.ts / houseKitDomain.ts / houseLotDomain.ts / placePropsDomain.ts — 건축

[S035] src/editor/tools/visionQueryTools.ts / houseVisionTools.ts / tileMetadataTools.ts — 시각 도구

[S036] src/editor/tools/questTools.ts / playTools.ts / battleTools.ts / historyTools.ts / discoveryTools.ts / aiDocTools.ts — 기타 도구

[S037] src/editor/panels/aiChatPanel.ts (3,086줄) — 채팅 패널 조립

[S038] src/editor/panels/aiTurnRunner.ts (837줄) — 턴 실행·적용 게이트

[S039] src/editor/panels/aiProposalCard.ts / aiProposalSummary.ts / aiProposalFusion.ts / aiChangePreview.ts — 제안·미리보기

[S040] src/editor/panels/aiComposer.ts / aiContextMeter.ts / aiDeckRail.ts / aiStudioShell.ts / aiRunSurface.ts — 컴포저·상태

[S041] src/editor/panels/aiSettingsModal.ts / aiAuthSettings.ts / aiConnectionStatus.ts / aiTemperatureMenu.ts — 설정

[S042] src/editor/panels/aiWorkPlanModal.ts / aiWorkPlanPages.ts / aiStickyChecklist.ts / aiHarnessModal.ts — 계획·관측

[S043] src/editor/panels/aiConversationLog.ts / aiConversationHistoryModal.ts / aiPreferenceMemorySettings.ts — 대화·기억 UI

[S044] src/editor/panels/databaseAiBar.ts / databaseAiGenerateDialog.ts / aiDatabaseGeneration.ts — DB AI

[S045] src/editor/panels/eventEditor/aiAssist.ts / showPictureAiField.ts / aiImageGenerateField.ts — 이벤트·이미지 AI

[S046] src/editor/panels/tilesetAiWorkspaceModal.ts / tilesetAiQuestionEditor.ts / tilesetAiMappingParser.ts / tilesetAiProposalModal.ts / tilesetAiProposalController.ts / tilesetAiCpenClient.ts — 타일셋 AI

[S047] src/editor/panels/clusterAiModal.ts / canvasAiWorkbench.ts / aiRegionTaskRunner.ts / regionTaskModal.ts — 범위 작업

[S048] src/editor/regionTask/runRegionTask.ts — 영역 작업 실행기

[S049] src/editor/aiAssistantBridge.ts / aiApplyCompletion.ts / aiBootIntent.ts / aiStudioMode.ts — 브리지·부팅

[S050] src/editor/detachedDraftMemory.ts / mapEditHistory.ts — 초안 사본·편집 이력

[S051] src/project/store.ts — 프로젝트 저장소(replace/update/persistCurrent/verifyPersistedRevision)

[S052] src/project/supabaseProjectSync.ts / projectCommitLog.ts — 원격 저장·커밋 기록

[S053] src/editor/characterAppearanceGeneration.ts — 외형 생성 컨트롤러

[S054] src/editor/panels/databaseModal.ts / databaseModalDirtySession.ts — DB 모달 롤백

[S055] scripts/lib/ohMyPiPiAiRuntime.ts — companion 어댑터(손실 경계)

[S056] scripts/lib/ohMyPiHttp.mjs / ohMyPiPiAi.mjs / aiAuthRuntime.ts / oh-my-pi-worker.ts — 서버 경로

[S057] scripts/lib/ohMyPiImageRuntime.ts / codexImageRuntime.ts — 이미지 서버

[S058] openwiki/editor-ai-panel.md (1,336줄) — 프로젝트 자체 문서(일부 stale)

[S059] openwiki/editor-ai-tools.md (875줄) — 도구 문서(148개·40상한 서술 stale)

[S060] openwiki/ai-workflow.md / ai-context-compaction.md / ai-tools-deprecation-roadmap.md — 워크플로·압축·폐기 문서

[S061] test/aiToolCallProtocol.test.ts / llmRetry.test.ts / ruleApprovalGate.test.ts / aiConfigMigration.test.ts — 존재 확인(실행 안 함)

[S062] 실행 실행: registry import (bun, exit 0) — 238/205/53/152/33/중복0

[S063] 실행 실행: eval에서 shouldCompact·describeContextUsage 직접 호출

[S064] 실행 캡처: 편집기 실제 화면 31장 (127.0.0.1:9841, 1440×900, 페이지 오류 0건). 대화·작업 계획·변경 카드는 로컬 예시 데이터를 실제 UI에 표시한 렌더링 증거이며, 실제 AI 실행 증거가 아닙니다.